

UNIVERSITÀ DEGLI STUDI DI NAPOLI
“FEDERICO II”



SCUOLA POLITECNICA E DELLE SCIENZE DI BASE
DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELLE
TECNOLOGIE DELL'INFORMAZIONE
D.I.E.T.I

TECHNICAL PROJECT
OF
FOUNDATIONS OF ROBOTICS

CONTROL AND ANALYSIS OF SCARA MANIPULATOR

Professor

Bruno Siciliano

Student

Paolo Antonio Casaretta

Matr. M54001568

ACADEMIC YEAR 2023/2024

Contents

Introduction	1
1 Kinematics Analysis	3
1.1 Direct Kinematics	3
1.2 Differential Kinematics	6
2 Manipulability Analysis	8
2.1 Velocity Manipulability Ellipsoid	8
2.2 Force Manipulability ellipsoid	9
2.3 Rappresentation and comparison between ellipsoids	10
3 Motion Planning	13
3.1 Path Planning	13
3.2 Trajectory Planning	15
4 Inverse Kinematics	18
4.1 Jacobian Inverse Algorithm	19
4.2 Jacobian Transpose Algorithm	22
5 Implementation of the CLIK Algorithm with a relaxed operational space component	24
6 Dynamic Model	30
7 Motion Control With a Payload	33
7.1 Second order CLIK algorithm with Jacobian inverse	33

7.2	Robust Control	36
7.3	Adaptive Control	42
7.4	Inverse Dynamics Control in Operational Space	46

List of Figures

1	SCARA manipulator	1
1.1	DH convention on SCARA manipulator	4
2.1	Manipulability Ellipsoids for q_1	10
2.2	Manipulability Ellipsoids for q_2	10
2.3	Manipulability Ellipsoids for q_3	10
2.4	Manipulability Ellipsoids for q_4	10
2.5	Manipulability Ellipsoids for q_5	11
2.6	Manipulability Ellipsoids for q_6 (Force)	11
2.7	Manipulability Ellipsoids for q_6 (Velocity)	11
3.1	Path in the operational space	15
3.2	Position,Velocity and Acceleration profiles	16
3.3	End Effector Positions	16
3.4	End Effector Velocities	17
3.5	End Effector Accelerations	17
4.1	Inverse kinematic algorithm with Jacobian inverse	19
4.2	Simulink scheme of CLIK algorithm with Jacobian inverse	20
4.3	Joint variables trend with CLIK algorithm with Jacobian inverse	20
4.4	Position error norm with CLIK algorithm with Jacobian inverse	21
4.5	Orientation error with CLIK algorithm with Jacobian inverse	21
4.6	Inverse kinematic algorithm with Jacobian transpose	22
4.7	Simulink scheme of CLIK algorithm with Jacobian transpose	22

4.8	Position error norm with CLIK algorithm with Jacobian transpose .	23
4.9	Orientation error with CLIK algorithm with Jacobian transpose . .	23
5.1	Simulink scheme of CLIK algorithm with Jacobian pseudo inverse .	26
5.2	Joint positions with Jacobian pseudo inverse	26
5.3	Position error norm with CLIK algorithm with Jacobian pseudo inverse	27
5.4	Orientation error with CLIK algorithm with Jacobian pseudo inverse	27
5.5	Manipulability measure for $K_0 = 10$	28
5.6	Joint positions with Jacobian pseudo inverse ($K_0 = 0$)	28
5.7	Manipulability measure for $K_0 = 0$	29
6.1	SCARA manipulator datas	31
7.1	Second order CLIK algorithm	34
7.2	Simulink scheme of second order CLIK algorithm	34
7.3	Position error norm with second order CLIK algorithm	35
7.4	Orientation error with second order CLIK algorithm	35
7.5	Robust control scheme	36
7.6	Space state diagram	37
7.7	Space state diagram with boundary layer	38
7.8	Simulink scheme for robust control	39
7.9		39
7.10	Joint position error for robust control	40
7.11	Joint torques with robust control ($\epsilon = 0.01$)	40
7.12	Position error norm with robust control	41
7.13	Orientation error with robust control	41
7.14	Joint torques with robust control ($\epsilon = 0.001$)	42
7.15	Adaptive control scheme	43
7.16	Simulink scheme for adaptive control	44
7.17	Position error norm with adaptive control	44
7.18	Orientation error with adaptive control	44

7.19 Joint torques with adaptive control	45
7.20 Joint position error with adaptive control	45
7.21 Inverse dynamics block scheme	47
7.22 Simulink scheme for inverse dynamics control	47
7.23 Position error norm with inverse dynamics control	48
7.24 Orientation error with inverse dynamics control	48
7.25 Joint torques with inverse dynamics control	49

Introduction

The SCARA manipulator (*Selective Compliance Assembly Robot Arm*) is a manipulator intended for the manipulation of small objects and it was developed to improve the velocity of operations like "pick and place" or to speed up and improve the assembly phases, because it shows high stiffness to vertical loads but it is compliant to horizontal ones.

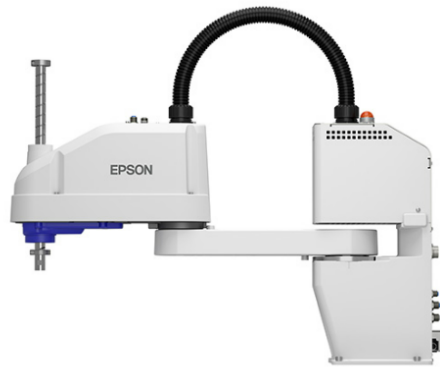


Figure 1: SCARA manipulator

In particular, it is made by four joints, of which:

- Two revolute joints that rotate around parallel axes which are orthogonal to the X-Y plane, allowing the manipulator to move in that plane.
- One prismatic joint that varies the height and therefore allows the motion along Z axis.

- Another revolute joint that allows the rotation of the end effector around its own axis.

The joints are driven by electric motors.

The goal of the project is to provide a kinematic and dynamic analysis of the manipulator through the manipulability measure of velocity and force, trajectory planning in the operational space, the study of the motion using algorithms for kinematic inversion with Jacobian inverse and traspose.

Chapter 1

Kinematics Analysis

Kinematics is the study of the motion without addressing the causes of it; we use it to describe the *pose*, position and orientation, of the end effector respect to joint variables.

A robotic manipulator can be represented as a kinematic chain of rigid bodies, called *links*, connected by the *joints*. Joints can be *prismatic*, if they allow a translational motion, or *revolute*, if they allow a rotational motion, and to each of them is associated with one degree of freedom (DOF); the DOF introduced by the joints are called *joint variables*.

One end of the kinematic chain is linked to a fixed base, while the other end is linked to the end effector, which allows the manipulation of objects in space; depending on how the chain is made, it can be *open*, when there is only one sequence of links that connects the two ends of the chain, or *closed*, there is a sequence of links forming a ring.

The purpose of direct kinematics is to identify the pose of the end effector knowing the joint variables, to obtain the function that binds the base frame to the end effector frame.

1.1 Direct Kinematics

To study the direct kinematics of a manipulator we need to identify the reference frames attached to each link and find the transformation that connects every frame to the following one.

To place the reference frames for each link *Denavit-Hartenberg* (DH) *convention* is used.

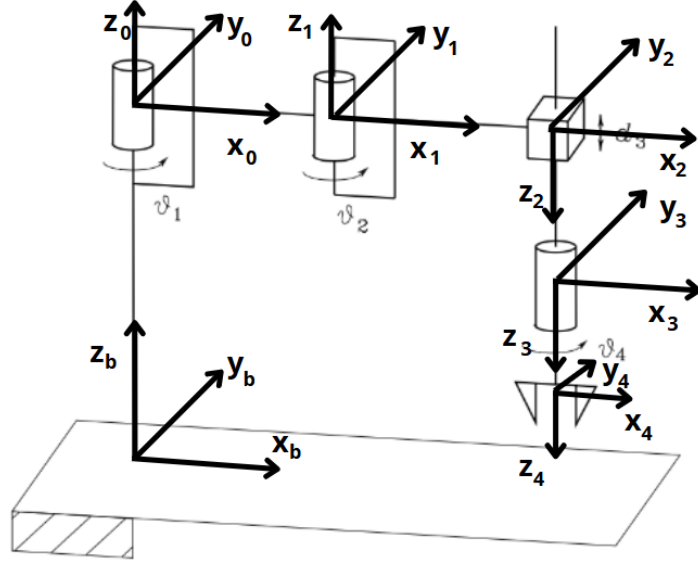


Figure 1.1: DH convention on SCARA manipulator

I have the vector of joint variables:

$$q = [\vartheta_1 \quad \vartheta_2 \quad d_3 \quad \vartheta_4]^T$$

And the DH table:

	a_i	α_i	d_i	ϑ_i
link 1	a_1	0	0	ϑ_1
link 2	a_2	π	0	ϑ_2
link 3	0	0	d_3	0
link 4	0	0	0	ϑ_4

I also want the transformation matrix that allow me to pass from one reference frame to the following one; using DH convention I have:

$$T_0^b = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad A_1^0(\vartheta_1) = \begin{bmatrix} c_1 & -s_1 & 0 & a_1 c_1 \\ s_1 & c_1 & 0 & a_1 s_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_2^1(\vartheta_2) = \begin{bmatrix} c_2 & s_2 & 0 & a_2 c_2 \\ s_2 & -c_2 & 0 & a_2 s_2 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad A_3^2(d_3) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad A_4^3(\vartheta_4) = \begin{bmatrix} c_4 & -s_4 & 0 & 0 \\ s_4 & c_4 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Here d_0 is the distance between the origin of frame b and the origin of frame 0, while c_i , s_i stand for $\cos \vartheta_i$ and $\sin \vartheta_i$ respectively.

If I want the transformation that connects frame b to the end effector frame, I simply multiply the transformation matrices:

$$T_4^0(q) = A_1^0(\vartheta_1)A_2^1(\vartheta_2)A_3^2(d_3)A_4^3(\vartheta_4) = \begin{bmatrix} c_{12}c_4 + s_{12}s_4 & -c_{12}s_4 + s_{12}c_4 & 0 & a_2c_{12} + a_1c_1 \\ s_{12}c_4 - c_{12}s_4 & -s_4s_{12} - c_{12}c_4 & 0 & a_2s_{12} + a_1s_1 \\ 0 & 0 & -1 & -d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_e^b(q) = T_0^b T_4^0(q) T_e^4 = \begin{bmatrix} c_{12}c_4 + s_{12}s_4 & -c_{12}s_4 + s_{12}c_4 & 0 & a_2c_{12} + a_1c_1 \\ s_{12}c_4 - c_{12}s_4 & -s_4s_{12} - c_{12}c_4 & 0 & a_2s_{12} + a_1s_1 \\ 0 & 0 & -1 & d_0 - d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Here instead c_{12} and s_{12} stand for $\cos(\vartheta_1 + \vartheta_2)$ and $\sin(\vartheta_1 + \vartheta_2)$. Notice that T_e^4 is the 4×4 identity matrix, because frame 4 coincides with the end effector frame.

1.2 Differential Kinematics

Differential kinematics is the relationship between the velocity of the joints $\dot{q}$ and the end effector velocity v_e :

$$v_e = J(q)\dot{q}$$

$$\begin{bmatrix} \dot{p}_e \\ \omega_e \end{bmatrix} = \begin{bmatrix} J_p(q) \\ J_o(q) \end{bmatrix} \dot{q}$$

The matrix $J(q)$ is called geometrical Jacobian; $J_p(q)$ is the submatrix that links joint velocities to the three components of linear velocity of the end effector, while $J_o(q)$ links joint velocities to the three components of the angular velocity.

The i^{th} column of geometrical Jacobian is given by:

$$\begin{bmatrix} J_{pi}(q) \\ J_{oi}(q) \end{bmatrix} = \begin{cases} \begin{bmatrix} z_{i-1} \\ 0 \end{bmatrix} & \text{for prismatic joint} \\ \begin{bmatrix} z_{i-1} \times (p_e - p_{i-1}) \\ z_{i-1} \end{bmatrix} & \text{for revolute joint} \end{cases}$$

So, for the SCARA manipulator the geometrical Jacobian is:

$$J(q) = \begin{bmatrix} z_0 \times (p_e - p_0) & z_1 \times (p_e - p_1) & z_2 & z_3 \times (p_e - p_3) \\ z_0 & z_1 & 0 & z_3 \end{bmatrix}$$

I get p_i from the last column of A_i^0 , and in the same way I get z_i from the third column of the same matrix:

$$A_2^0 = A_1^0 A_2^1 = \begin{bmatrix} c_{12} & s_{12} & 0 & a_2 c_{12} + a_1 c_1 \\ s_{12} & -c_{12} & 0 & a_2 s_{12} + a_1 s_1 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad A_3^0 = A_2^0 A_3^2 = \begin{bmatrix} c_{12} & s_{12} & 0 & a_2 c_{12} + a_1 c_1 \\ s_{12} & -c_{12} & 0 & a_2 s_{12} + a_1 s_1 \\ 0 & 0 & -1 & -d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$p_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad p_1 = \begin{bmatrix} a_1 c_1 \\ a_1 s_1 \\ 0 \end{bmatrix} \quad p_3 = \begin{bmatrix} a_2 c_{12} + a_1 c_1 \\ a_2 s_{12} + a_1 s_1 \\ -d_3 \end{bmatrix} \quad p_e = p_4 = p_3$$

$$z_0 = z_1 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad z_2 = z_3 = \begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix}$$

$$J(q) = \begin{bmatrix} -a_2 s_{12} - a_1 s_1 & -a_2 s_{12} & 0 & 0 \\ a_2 c_{12} + a_1 c_1 & a_2 c_{12} & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} -a_2 s_{12} - a_1 s_1 & -a_2 s_{12} & 0 & 0 \\ a_2 c_{12} + a_1 c_1 & a_2 c_{12} & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 1 & 1 & 0 & -1 \end{bmatrix}$$

Every column is the contribution of motion that each joint gives to the end effector.

Chapter 2

Manipulability Analysis

There are algebraic tools that allow to define, in the fixed configuration q , the attitude of the manipulator to exert velocity and force; these tools are called *velocity manipulability ellipsoid* and *force manipulability ellipsoid*, respectively.

2.1 Velocity Manipulability Ellipsoid

Consider a sphere in the joint velocity space, a sphere of all possible joint velocities applied on the manipulator; this sphere has radius 1:

$$\dot{q}^T \dot{q} = 1$$

Using the relationship between joint velocities and end effector velocity it is possible to derive an expression for the velocity manipulability ellipsoid:

$$v_e = J(q)\dot{q} \rightarrow \dot{q} = J(q)^\dagger v_e$$

$$J^\dagger = J^T (J J^T)^{-1} \rightarrow$$

$$\dot{q}^T = v_e^T J^{\dagger T}(q)$$

Doing the substitutions I obtain:

$$v_e^T (J J^T)^{-1} J J^T (J J^T)^{-1} v_e = 1 \rightarrow$$

$$v_e^T (J(q) J^T(q))^{-1} v_e = 1$$

So, given a sphere in the joint velocity space I have an ellipsoid in the end effector velocity space, where the direction of the axes of the ellipsoid is given by the eigenvectors of $J(q)J^T(q)$, while the dimensions of the axes are given by the eigenvalues of the same matrix.

The direction of the major axis shows the direction along which the manipulator can move at higher velocities; on the other hand, the direction of the minor axis shows the direction where I have lower velocities on the end effector.

2.2 Force Manipulability ellipsoid

Using kineto-static duality, it is possible to describe the manipulability of the structure not only with velocities, but also with forces.

in the same way as before, consider a sphere in the space of joint torques:

$$\tau^T \tau = 1$$

As for the velocity, it is possible to derive an expression for the force ellipsoid:

$$\tau = J^T \gamma_e$$

$$\gamma_e^T (J(q)J(q)^T) \gamma_e = 1$$

γ_e is called *wrench*, and it represents forces and torques at the end effector.

2.3 Representation and comparison between ellipsoids

Here are reported simulations of velocity and force manipulability ellipsoids for certain configurations of the manipulator:

$$q_1 = \begin{bmatrix} -\frac{\pi}{4} & \frac{\pi}{2} & 0 & 0 \end{bmatrix}^T \quad q_2 = \begin{bmatrix} \frac{\pi}{4} & -\frac{\pi}{2} & 0 & 0 \end{bmatrix}^T \quad q_3 = \begin{bmatrix} -\frac{\pi}{4} & -\frac{\pi}{2} & 0 & 0 \end{bmatrix}^T$$

$$q_4 = \begin{bmatrix} \frac{\pi}{4} & \frac{\pi}{2} & 0 & 0 \end{bmatrix}^T \quad q_5 = \begin{bmatrix} -\frac{\pi}{6} & \frac{\pi}{6} & 0 & 0 \end{bmatrix}^T \quad q_6 = \begin{bmatrix} \frac{\pi}{6} & 0 & 0 & 0 \end{bmatrix}^T$$

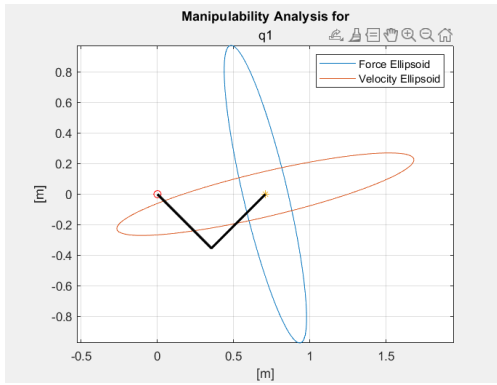


Figure 2.1: Manipulability Ellipsoids for q_1

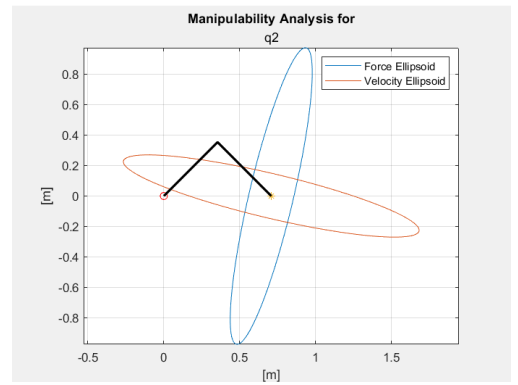


Figure 2.2: Manipulability Ellipsoids for q_2

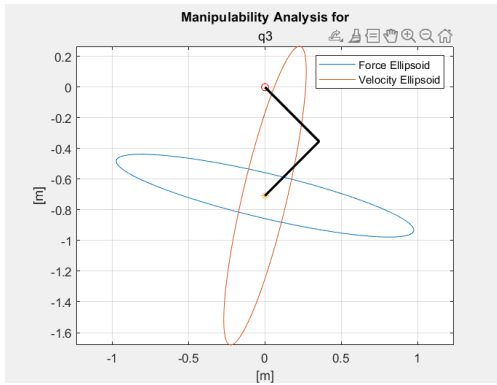


Figure 2.3: Manipulability Ellipsoids for q_3

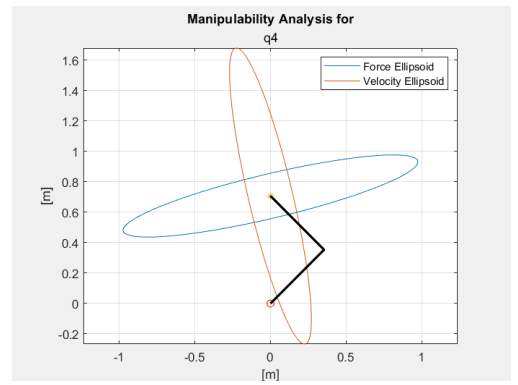


Figure 2.4: Manipulability Ellipsoids for q_4

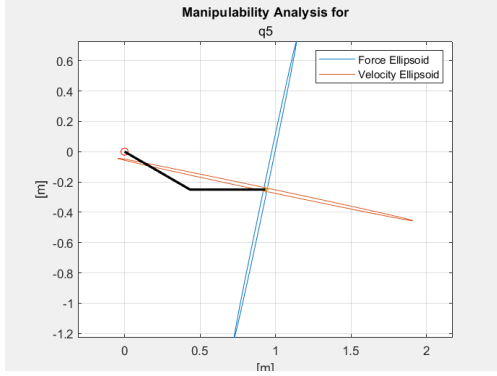


Figure 2.5: Manipulability Ellipsoids for q_5

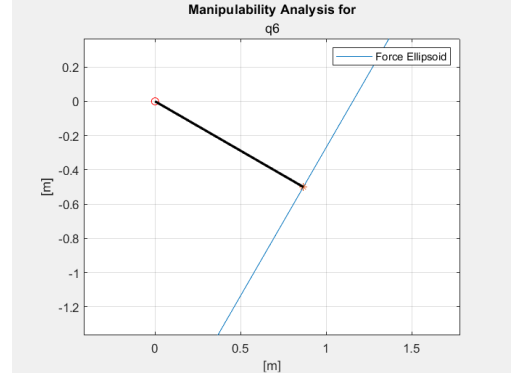


Figure 2.6: Manipulability Ellipsoids for q_6 (Force)

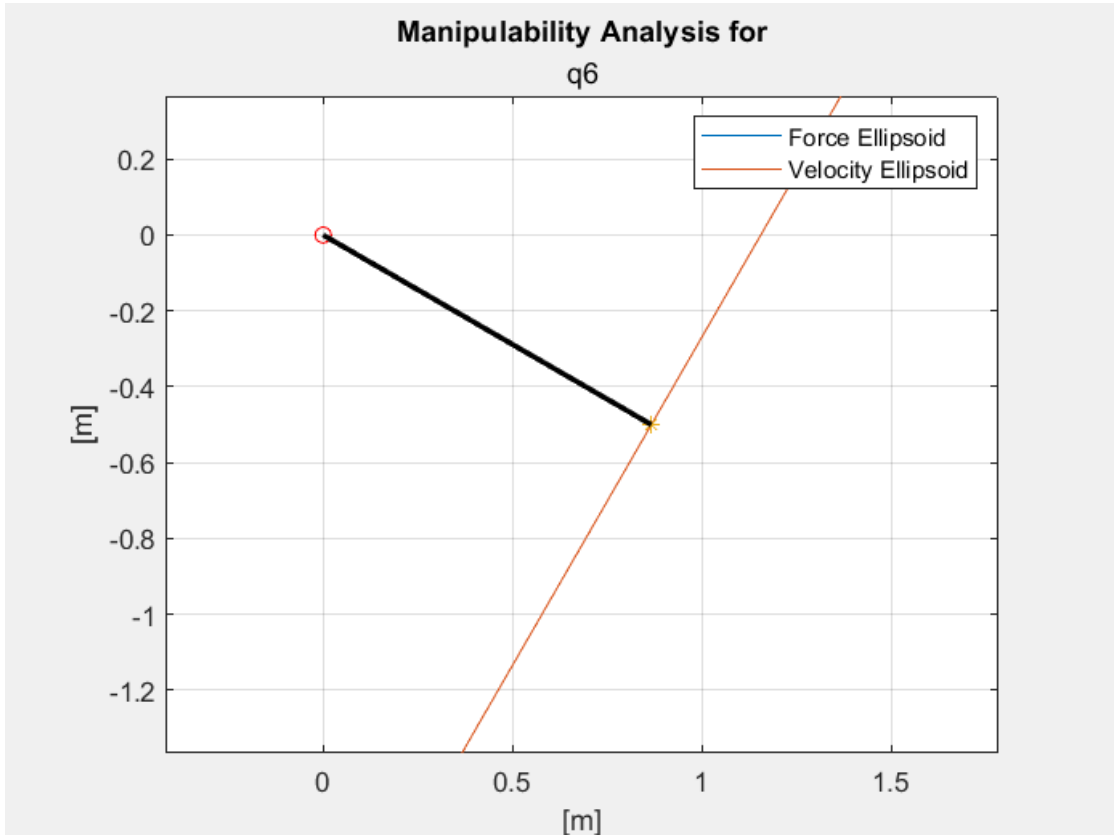


Figure 2.7: Manipulability Ellipsoids for q_6 (Velocity)

The first two configurations are elbow down and elbow up. In the third and fourth configurations it may seem that the ellipsoids switched place, but in reality the manipulator simply moved lower or higher.

In the fifth configuration it is shown how, when the manipulator stretches out, both ellipsoids become narrower, which means that manipulability, both in force and velocity, is reduced.

Notice that only the first two joints were considered, the supporting structure, because the third joint only contributes to the traslation along Z axis and the fourth one only to the orientation of the end effector.

When the manipulator becomes singular, the ellipsoids degenerate into a straight line, which means that the manipulator can exert force and velocity only along the direction of the ellipsoids.

Mathematically, this happens because when the manipulator gets singular JJ^T for the force ellipsoid and $(JJ^T)^{-1}$ for the velocity ellipsoid have an eigenvalue equal to 0.

To have an esteem of the manipulation ability into a given configuration, reference is made to a quantity called *manipulability measure*, defined as:

$$w(q) = \sqrt{\det(J(q)J(q)^T)}$$

For the chosen configuration, $w(q)$ is equal to:

$$w(q_1) = w(q_2) = w(q_3) = w(q_4) = 0.2500$$

$$w(q_5) = 0.1250$$

Manipulability is maximum when the arms are ortogonal, while it decreases when the structure gets closer to singularity. To study when a manipulator is singular, it is needed to determine when the determinant of the Jacobian is 0; for the SCARA manipulator:

$$\det(J) = a_1 a_2 s_2$$

So the manipulator becomes singular when $\vartheta_2 = 0$, $\vartheta_2 = \pi$, so when the supporting structure is fully stretched out (elbow singularity).

Chapter 3

Motion Planning

A robotic manipulator must perform a task in the operational space that involves its movement along a given path, with desired velocities and accelerations; plan a trajectory means generating reference inputs for the motion control system, which are a series of poses that the end effector will have to describe over time.

The *path* is the pure geometric description of the motion through the set of points; if to this path is added a time law, something related to velocity, how fast we move, this is called *trajectory*.

3.1 Path Planning

In this project a path consisting of 20 points was created, of which 4 are via points (highlighted in green); the path has straight and circular sections:

Points	P_x	P_y	P_z	α
P ₁	0.5	0.5	1	0
P ₂	0.4	0.5	1	$-\frac{\pi}{8}$
P ₃	0.3	0.5	1	$-\frac{\pi}{8}$
P ₄	0.2	0.5	1	$-\frac{\pi}{8}$
P ₅	0.2	0.3	1	$-\frac{\pi}{8}$
P ₆	0.3	0.3	1	$-\frac{\pi}{8}$
P ₇	0.35	0.35	1	$\frac{\pi}{6}$
P ₈	0.35	0.35	0.7	$\frac{\pi}{6}$
P ₉	0.4	0.45	0.7	$\frac{\pi}{6}$
P ₁₀	0.5	0.45	0.7	0
P ₁₁	0.55	0.45	0.7	0
P ₁₂	0.55	0.35	0.7	0
P ₁₃	0.55	0.3	0.7	0
P ₁₄	0.55	0.25	0.7	0
P ₁₅	0.5	0.25	0.7	0
P ₁₆	0.5	0.25	0.85	0
P ₁₇	0.45	0.25	0.85	0
P ₁₈	0.45	0.3	0.85	0
P ₁₉	0.45	0.35	0.85	0
P ₂₀	0.45	0.4	0.85	0
P ₂₁	0.45	0.45	0.85	0
P ₂₂	0.5	0.5	1	0

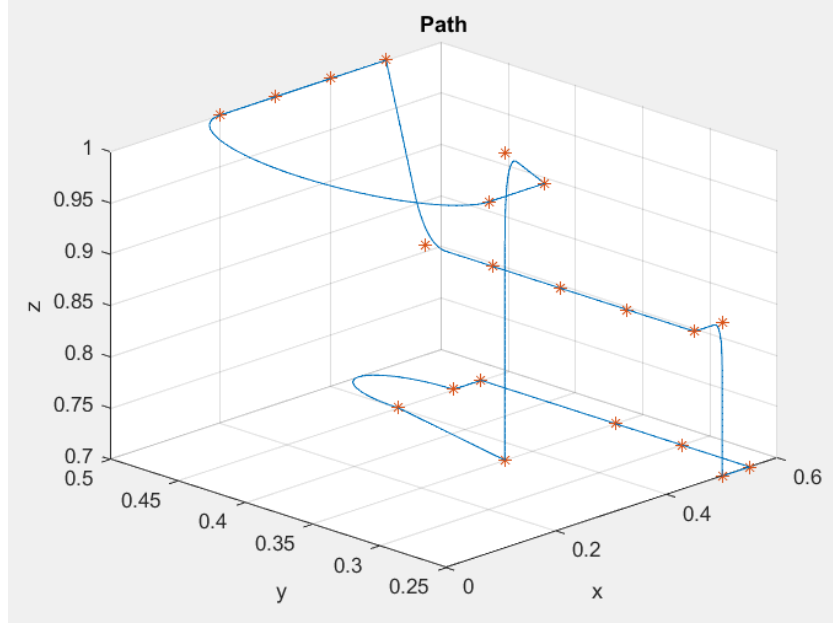


Figure 3.1: Path in the operational space

3.2 Trajectory Planning

After planning the path, the trajectory must be planned, so a time law must be defined, which means to define a sequence of velocities and accelerations that the end effector must perform while tracking the path.

To associate a time law with the path, it is parameterized with the function $s(t)$ which, in this case, is defined by a trapezoidal motion law.

This trapezoidal velocity profile is characterized by the fact that the velocity has a positive sloping section at which there will be constant acceleration, a no sloping section, called "at cruising speed", at which there will be zero acceleration and a section with a negative slope in correspondence of which there will be constant deceleration.

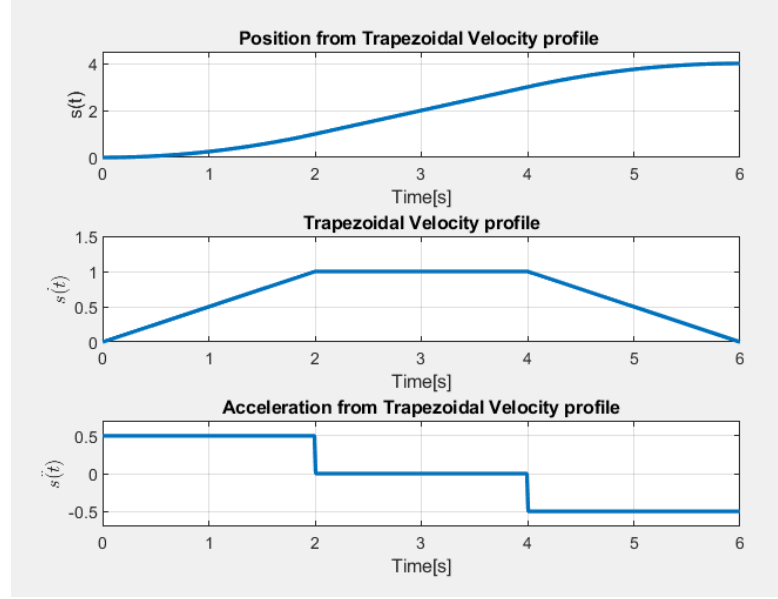


Figure 3.2: Position, Velocity and Acceleration profiles

Then the trend over time of the poses, speeds and accelerations at the end effector is shown.

The simulation of the trajectory takes place in 108 seconds, and the displacement between one point and the next one takes place in 6 seconds.

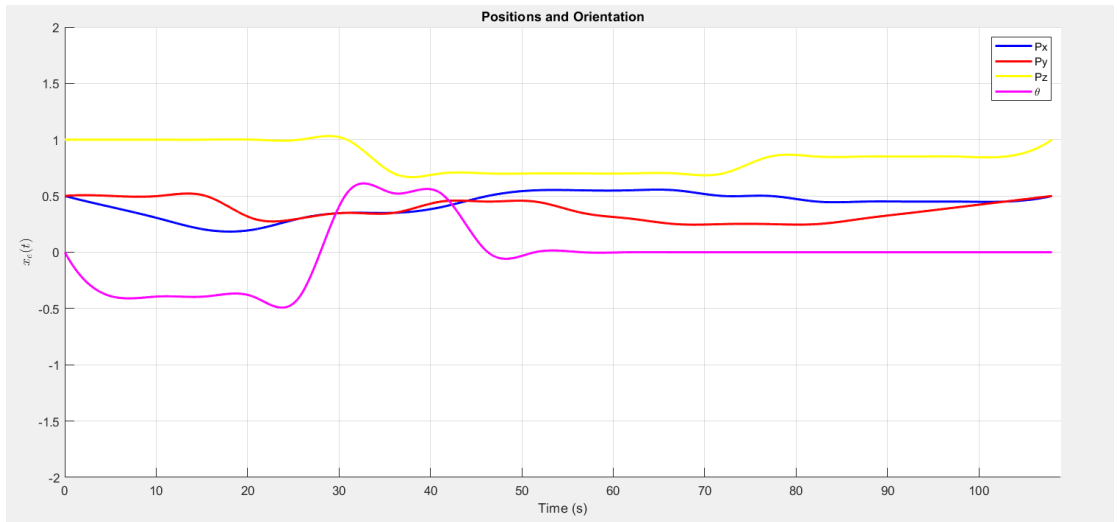


Figure 3.3: End Effector Positions

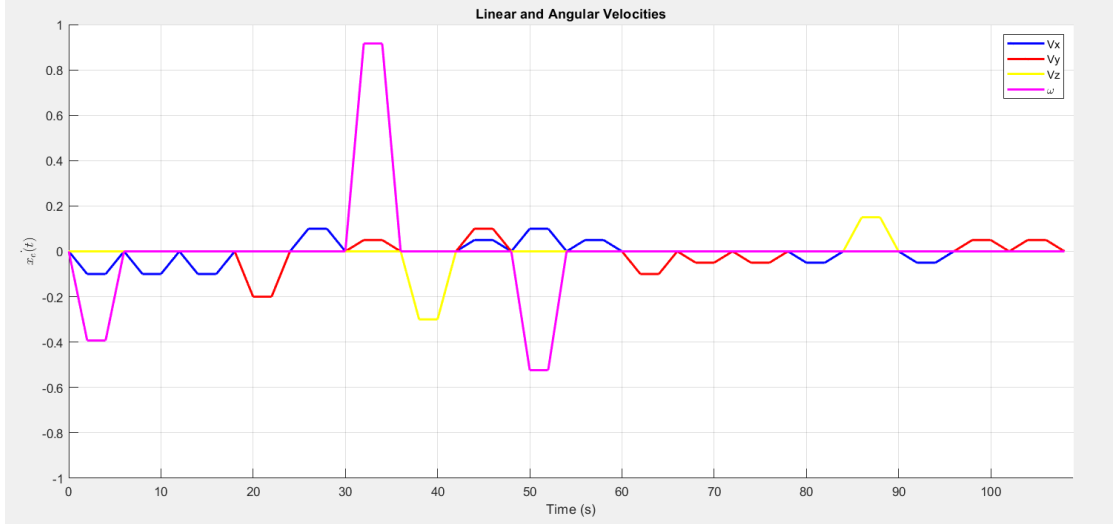


Figure 3.4: End Effector Velocities

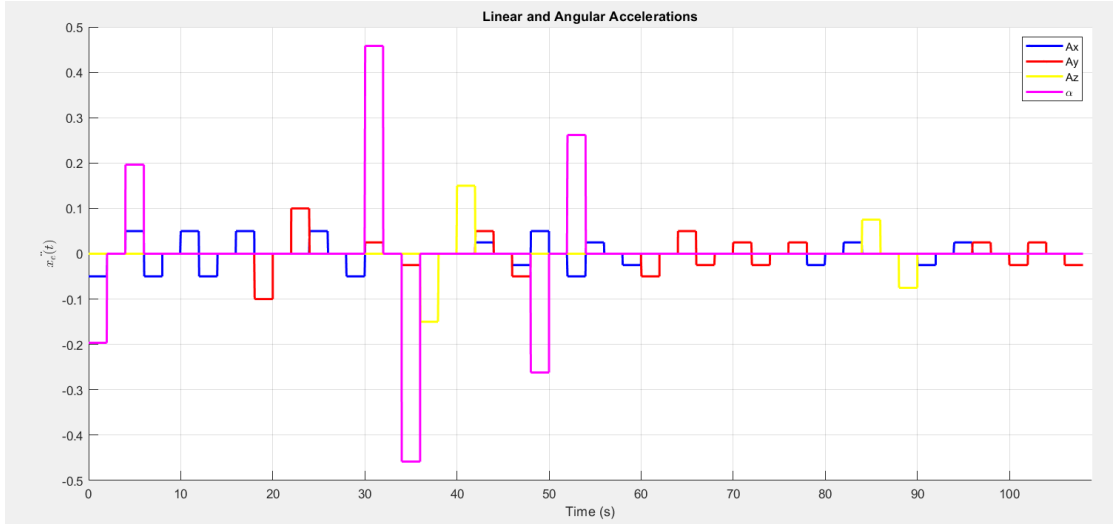


Figure 3.5: End Effector Accelerations

From the graphs, it can be seen that the planning with a trapezoidal velocity profile has a problem: it is possible to guarantee the continuity of velocities but not of accelerations, and to obtain it continuity constraints on intermediate sections must be imposed.

Chapter 4

Inverse Kinematics

Direct kinematics define the functional relationships between the position and the orientation of the end effector, in the operational space, and the position and orientation of the joints, in the joint space, while inverse kinematics consists of determining the joint variables once the pose assigned to the end effector is known.

Since the direct kinematic equation is strongly non linear, problems may arise in the solution of the inverse kinematic problem; to solve the problem differential kinematic can be used.

$$v_e = J(q)\dot{q} \rightarrow \dot{q} = J^{-1}(q)v_e$$

Integrating $\dot{q}$ using Euler numerical integration rule $q(t)$ is obtained.

However, there is still an error between end effector's actual position and the desired one:

$$e = x_d - x_e$$

$$\dot{e} = \dot{x}_d - \dot{x}_e = \dot{x}_d - J_A(q)\dot{q}$$

To bring the error to 0, CLIK (Closed Loop Inverse Kinematics) algorithms are used, and there are 2 types of them: CLIK algorithm with Jacobian inverse and CLIK algorithm with Jacobian transpose.

4.1 Jacobian Inverse Algorithm

For CLIK algorithm with Jacobian inverse, to make the error converge to 0, a joint variable vector that depends on the error must be chosen:

$$\dot{q} = J_A^{-1}(q)(\dot{x}_d(t) + Ke)$$

Where $\dot{x}_d(t)$ is the desired velocity at the end effector, K is a positive definite matrix, usually diagonal, and e is the error between the actual pose and the desired one.

Replacing $\dot{q}$ expression into the expression of the error, the following equation is obtained:

$$\dot{e} + Ke = 0$$

Which is a linear system globally asymptotically stable.

The error goes to 0 with a speed directly proportional to K eigenvalues, which in this case has the following expression:

$$K = \begin{pmatrix} 100 & 0 & 0 & 0 \\ 0 & 100 & 0 & 0 \\ 0 & 0 & 100 & 0 \\ 0 & 0 & 0 & 100 \end{pmatrix}$$

The block diagram of a CLIK algorithm with Jacobian inverse looks like this:

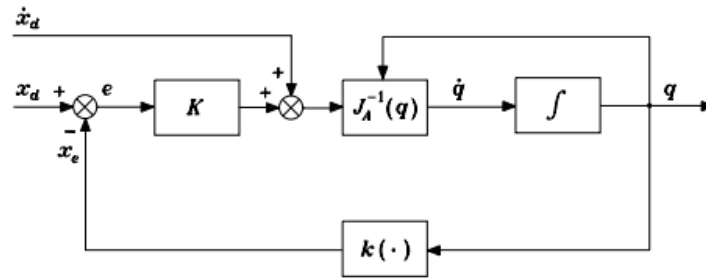


Figure 4.1: Inverse kinematic algorithm with Jacobian inverse

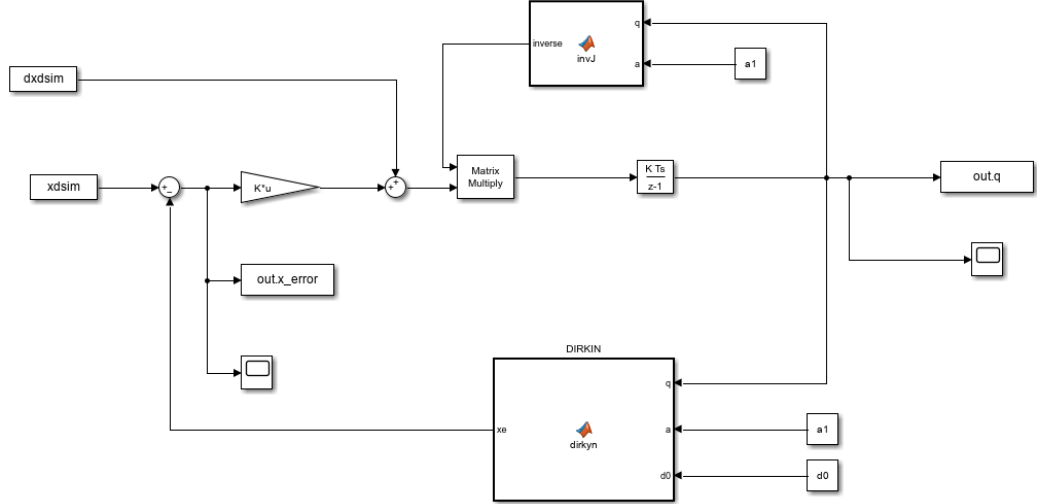


Figure 4.2: Simulink scheme of CLIK algorithm with Jacobian inverse

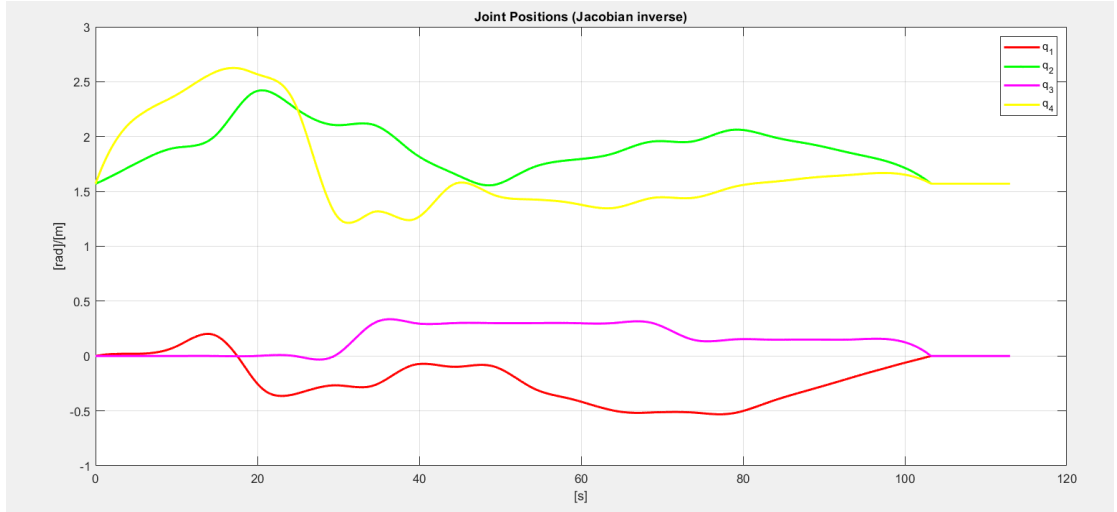


Figure 4.3: Joint variables trend with CLIK algorithm with Jacobian inverse

For the norm of the position error it can be seen that at steady state the error is 0 while during the transient it has a magnitude of 10^{-3} ; also the orientation error can be seen, and it is null at steady state:

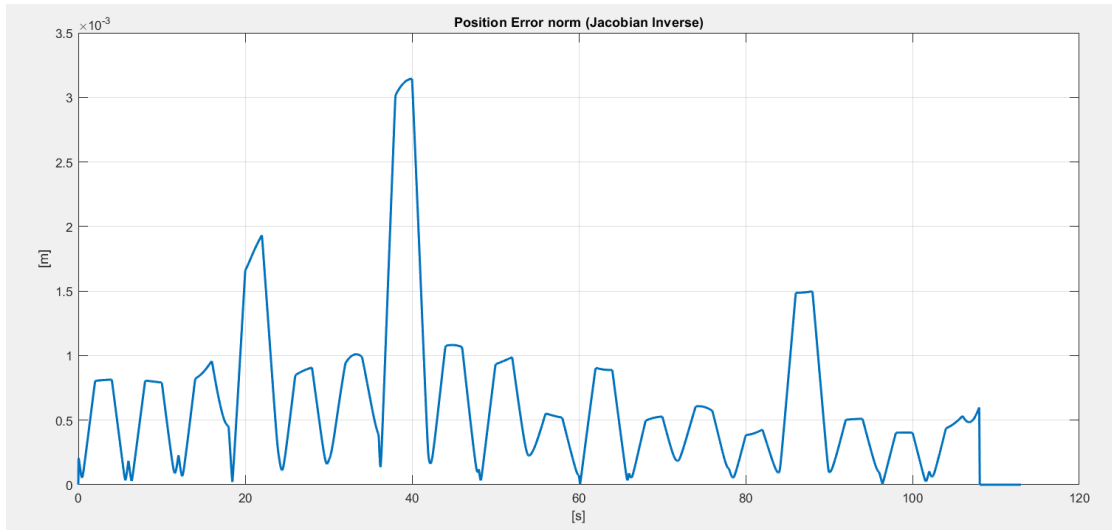


Figure 4.4: Position error norm with CLIK algorithm with Jacobian inverse

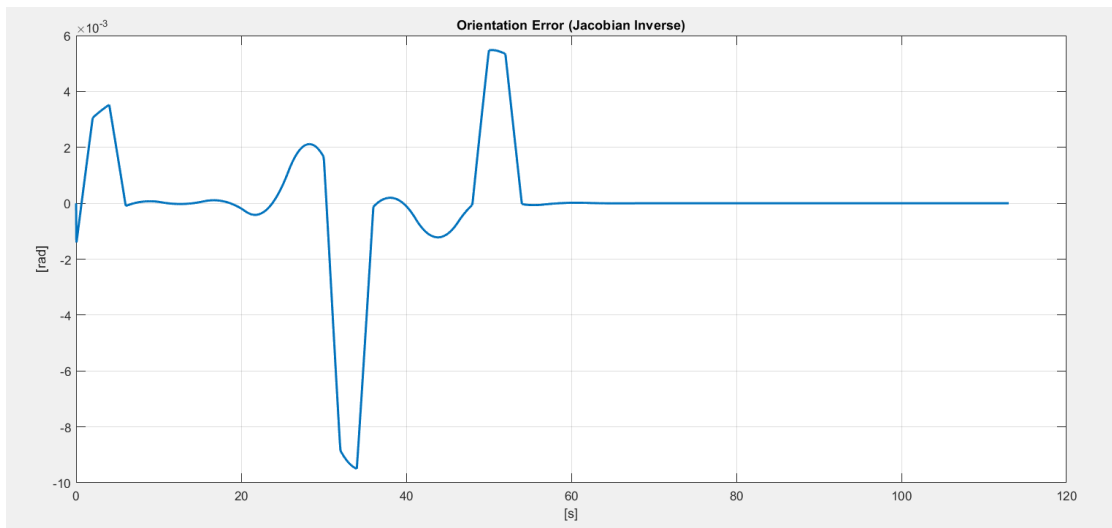


Figure 4.5: Orientation error with CLIK algorithm with Jacobian inverse

4.2 Jacobian Transpose Algorithm

In case of singularities, it is not possible to apply the Jacobian inverse algorithm, so it is needed an algorithm that ensures the convergence to 0 of the error without linearizing its expression. In this case, by means of Lyapunov's direct method, an expression of velocity at joints in terms of Jacobian transpose is obtained:

$$\dot{q}(t) = J_A^T(q) K e$$

Where the matrix K is chosen like this:

$$K = \begin{pmatrix} 200 & 0 & 0 & 0 \\ 0 & 200 & 0 & 0 \\ 0 & 0 & 200 & 0 \\ 0 & 0 & 0 & 200 \end{pmatrix}$$

The block diagram of a CLIK algorithm with Jacobian transpose looks like this:

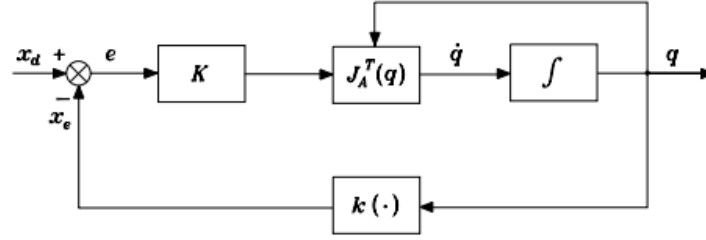


Figure 4.6: Inverse kinematic algorithm with Jacobian transpose

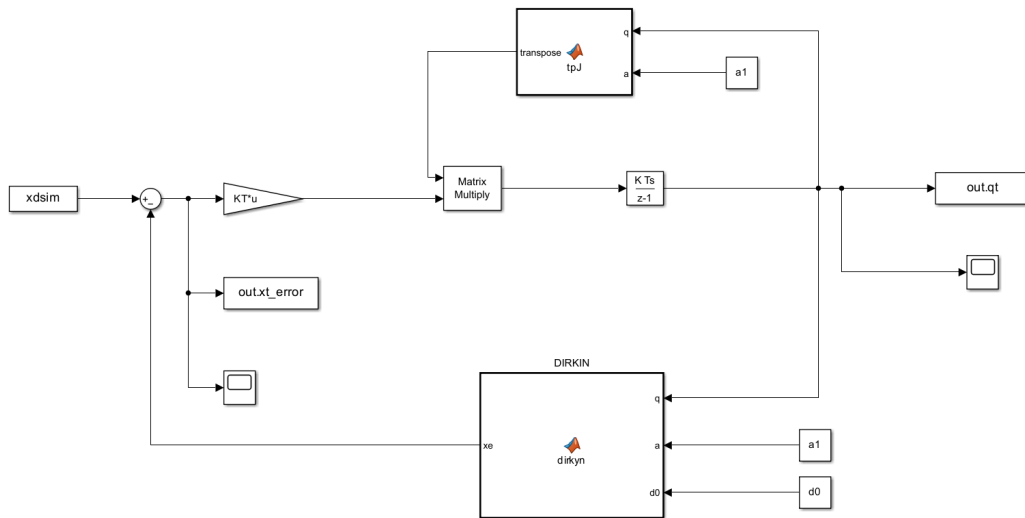


Figure 4.7: Simulink scheme of CLIK algorithm with Jacobian transpose

The trend for joint variables is exactly the same as before.

Although the Jacobian transpose CLIK algorithm has a lower computational burden compared to the Jacobian inverse one, it can be seen that both position and orientation error exhibit a worse transient behavior respect to the errors of the Jacobian inverse algorithm, despite the fact that a matrix K with higher values was chosen:

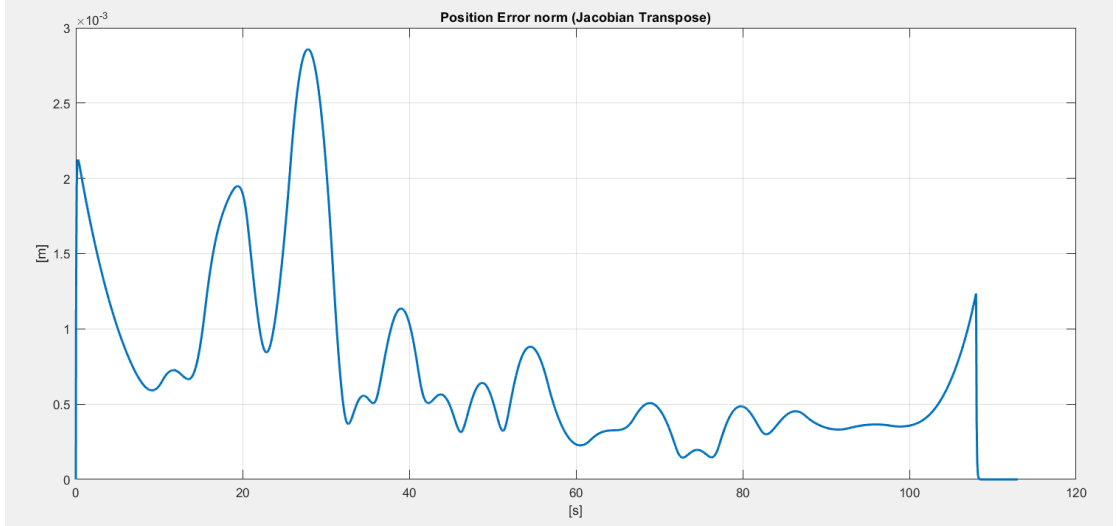


Figure 4.8: Position error norm with CLIK algorithm with Jacobian transpose

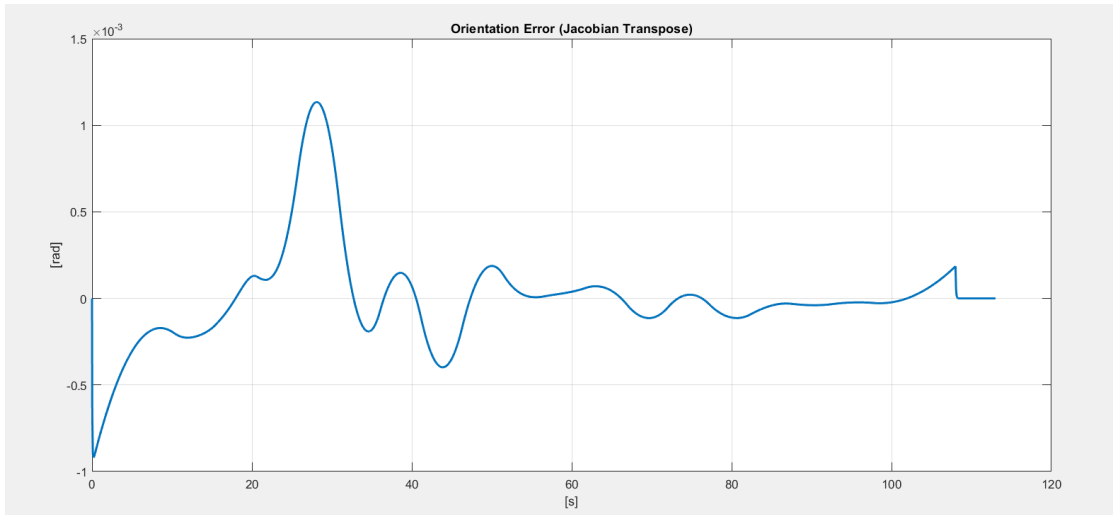


Figure 4.9: Orientation error with CLIK algorithm with Jacobian transpose

Chapter 5

Implementation of the CLIK Algorithm with a relaxed operational space component

Relaxing an operational space component means to not impose the value of one of the three axes of the operational space, and then implement the CLIK algorithm using Jacobian pseudo-inverse, in order to optimize the dexterity of the manipulator.

This choice makes the manipulator functionally *redundant*, which means that (Given that in this case the y direction is relaxed) the second row of the Jacobian can be ignored, making it a 3×4 matrix.

The pseudo-inverse of the Jacobian is defined as:

$$J^\dagger = J^T(JJ^T)^{-1}$$

And the following relationship applies:

$$\dot{q} = J^\dagger v_e$$

Given a redundant manipulator, if $\dot{q}^*$ is a solution, so it is $\dot{q}^* + P\dot{q}_0$, where $\dot{q}_0$ is an arbitrary non null vector and P is the projector into Jacobian's null space; $P = I - J^\dagger J$, so the previous equation can be modified as follows:

$$\dot{q} = J^\dagger v_e + (I - J^\dagger J)\dot{q}_0$$

This implies that, given $v_e = 0$, there are values of $\dot{q}$, namely $\dot{q}_0$, that don't alter the pose of the end effector, but can change the position of other joints, called *internal motions of the structure*. In this way, for example, a better pose to perform the next task can be achieved, or the manipulator can be reoriented to avoid singularities or obstacles.

$\dot{q}_0$ is chosen as:

$$\dot{q}_0 = K_0 \left(\frac{\partial w(q)}{\partial q} \right)^T$$

Where $w(q)$, is the same manipulability measure defined in chapter 2, and can be chosen to satisfy different conditions. In this case it will be chosen so to move away from singularities, because the task asks to optimize the dexterity. For this reason, the manipulability measure $w(q)$ is chosen in such a way that for $K_0 > 0$ the manipulator approaches/moves away from the singularity with more dexterous postures:

$$w(q) = \sqrt{\det(J(q)J^T(q))}$$

A computationally simpler $w(q)$ can be chosen; because it has been seen, in chapter 2, that the manipulator goes singular when $\vartheta_2 = 0, \pi$, so manipulability measure is linked to the second joint variable, and so $w(q)$ will be chosen as follows:

$$w(q) = \sin^2(\vartheta_2)$$

So that by maximizing it, the manipulator will steer away from singularities.

The control scheme to be implemented is represented by the following equation:

$$\dot{q} = J^\dagger(\dot{x}_d + Ke) + (I - J^\dagger J)\dot{q}_0^*$$

The matrix K was chosen like this:

$$K = \begin{pmatrix} 100 & 0 & 0 \\ 0 & 100 & 0 \\ 0 & 0 & 100 \end{pmatrix}$$

While $K_0 = 10$:

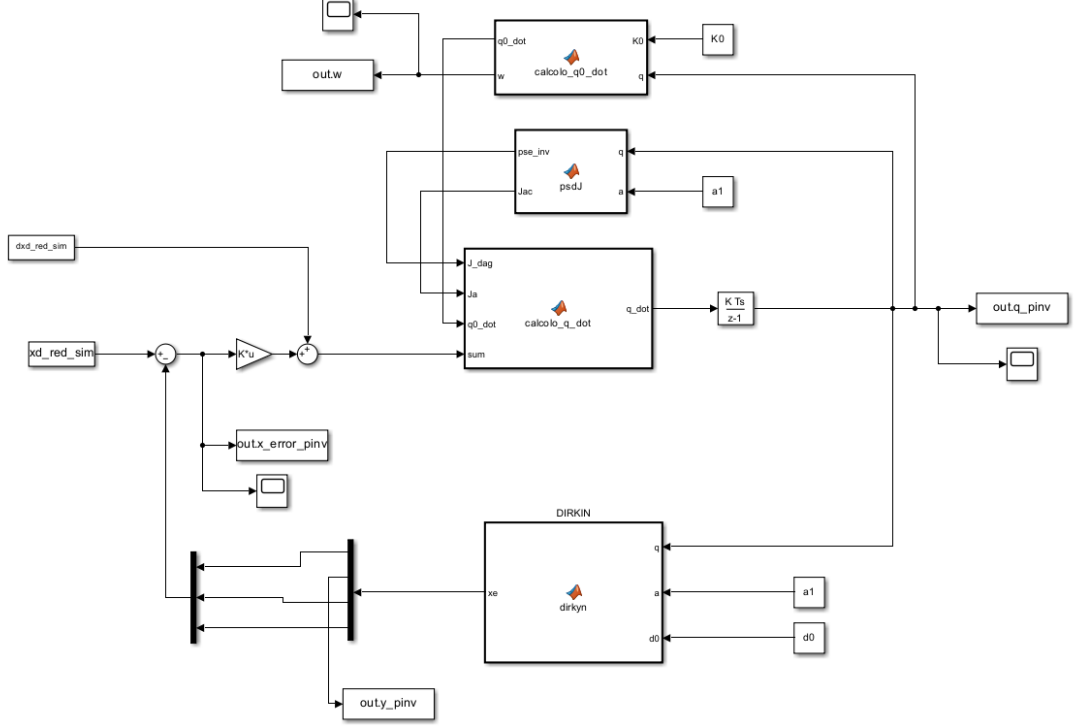


Figure 5.1: Simulink scheme of CLIK algorithm with Jacobian pseudo inverse

Here are shown the trends of joint positions, errors and manipulability measure:

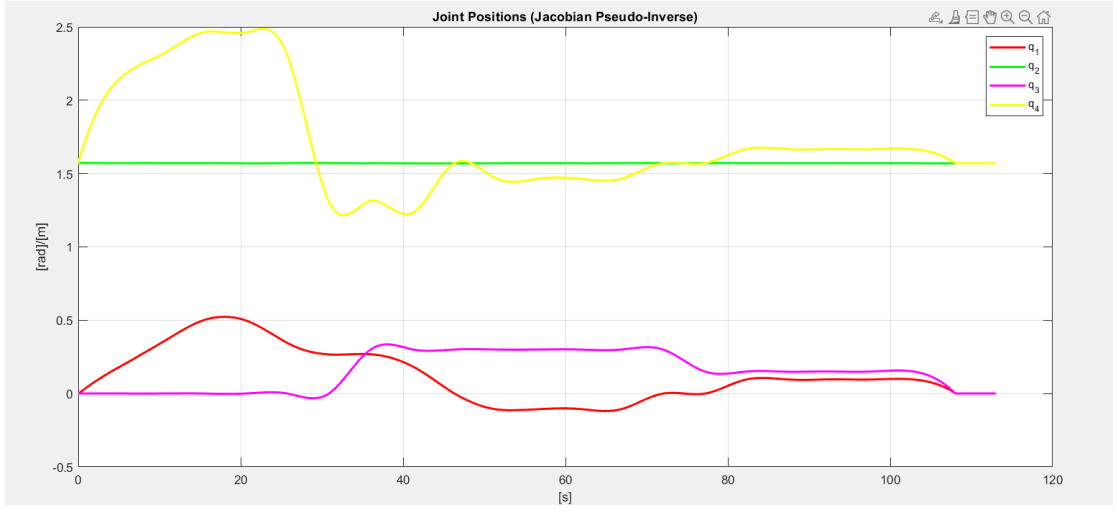


Figure 5.2: Joint positions with Jacobian pseudo inverse

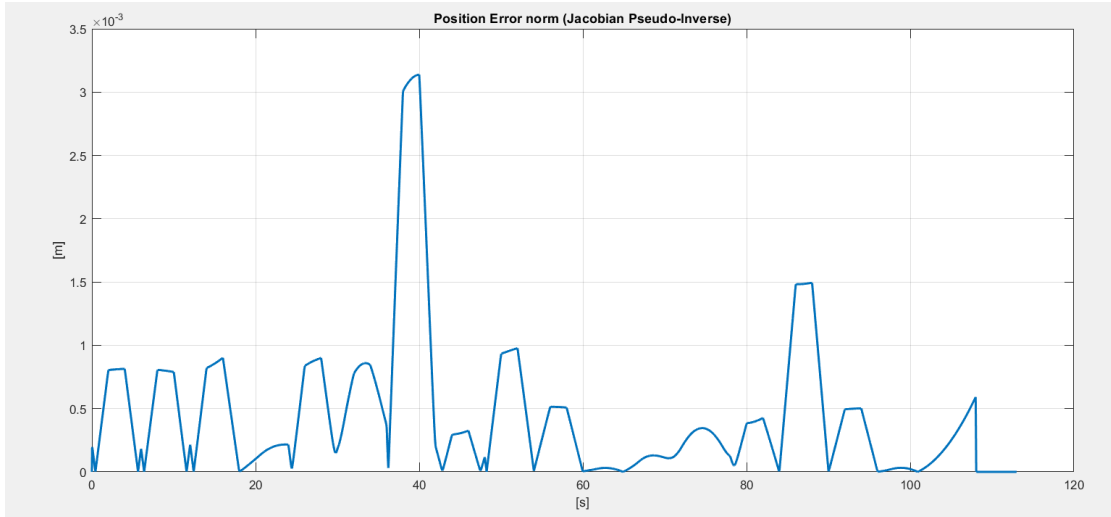


Figure 5.3: Position error norm with CLIK algorithm with Jacobian pseudo inverse

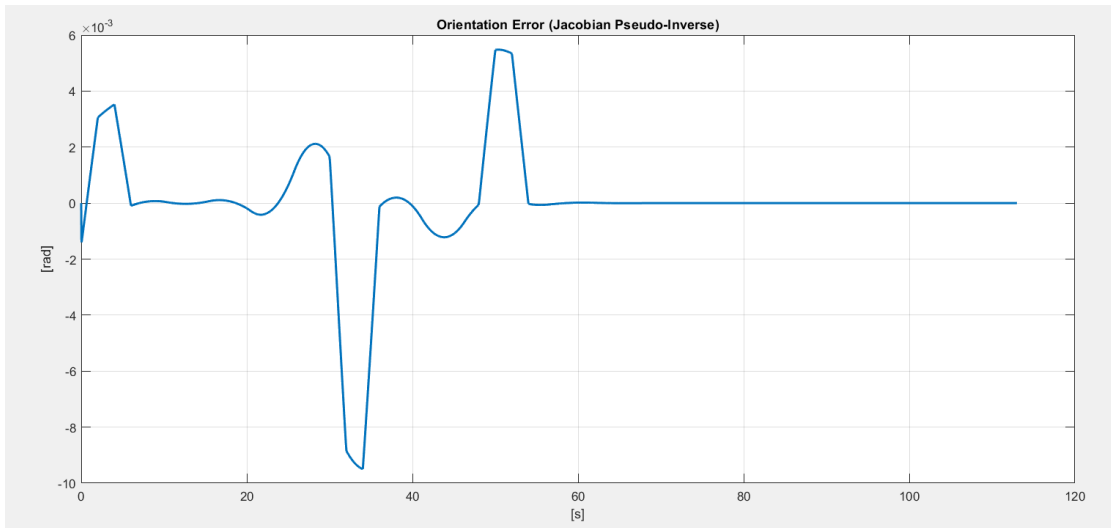


Figure 5.4: Orientation error with CLIK algorithm with Jacobian pseudo inverse

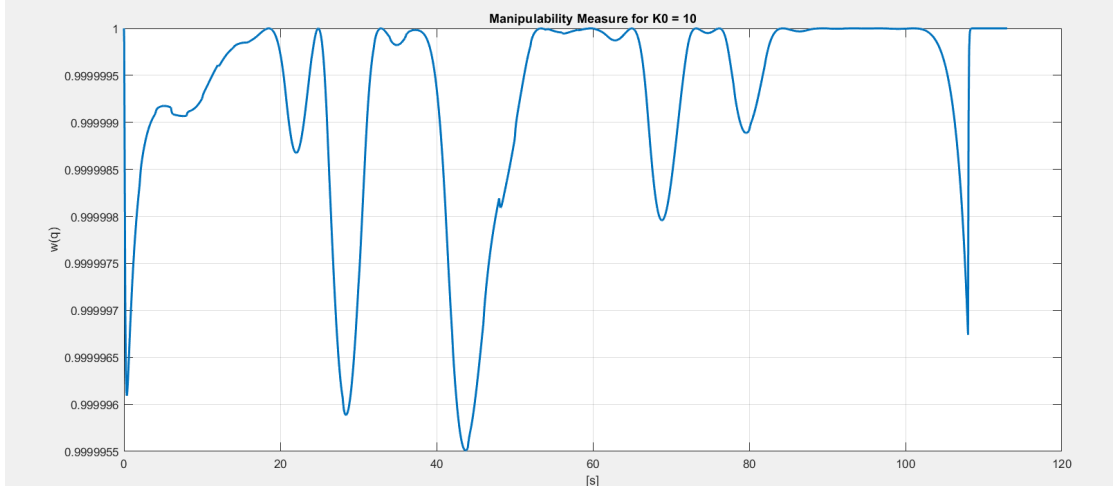


Figure 5.5: Manipulability measure for $K_0 = 10$

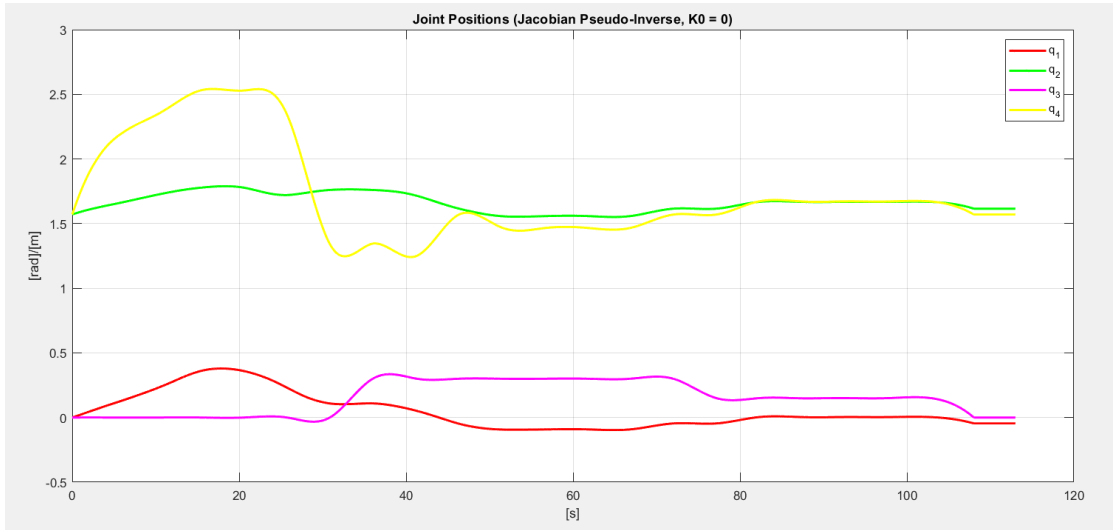


Figure 5.6: Joint positions with Jacobian pseudo inverse ($K_0 = 0$)

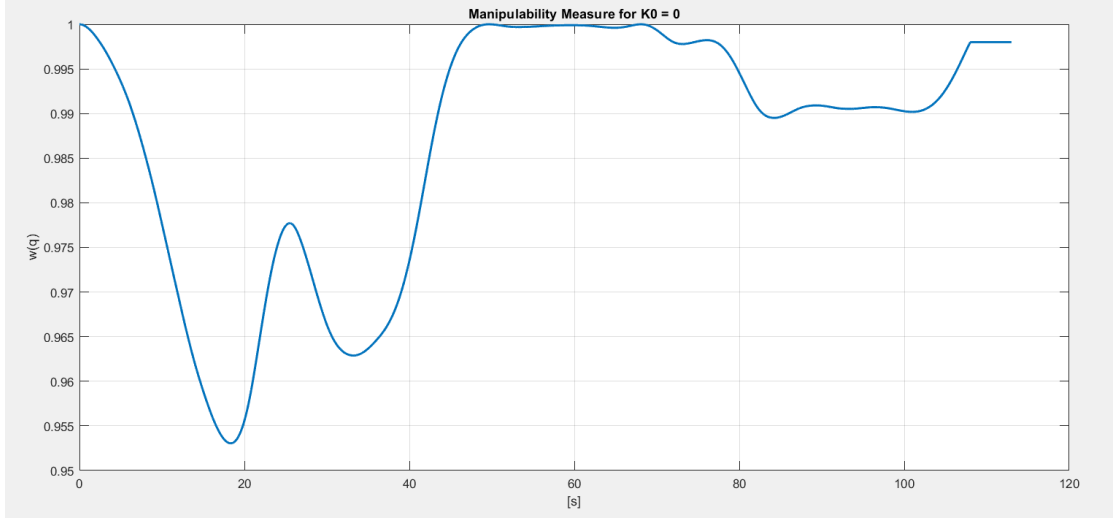


Figure 5.7: Manipulability measure for $K_0 = 0$

When $K_0 = 0$ it can be noticed that when q_2 gets near singularity the manipulability measure diminishes, and it is also not as good as when $K_0 = 10$. With $K_0 = 10$, it can be seen that q_1 and q_4 change to allow q_2 to move away from singularity.

Chapter 6

Dynamic Model

The dynamic model is derived with the *Lagrange formulation* of the SCARA manipulator. The dynamic model ties together the joint forces and the motion of the structure. The approximation that will be done is considering that the angular velocity of each rotor is due to its own spinning. The equation of the dynamic model is as follows:

$$B(q)\ddot{q} + C(q, \dot{q})\dot{q} + F_v\dot{q} + g(q) = \tau$$

Where:

- $B(q)$ is a positive definite matrix, the inertia matrix, $[n \times n]$.
- $C(q, \dot{q})$ is the matrix that accounts for Coriolis and centrifugal effects, $[n \times n]$.
- F_v is the diagonal matrix which contains the coefficients of viscous friction, $[n \times n]$.
- $g(q)$ is the gravitational terms vector, $[n \times 1]$.
- τ is the vector of the torques at the joints, $[n \times 1]$.

The inertia matrix is defined as:

$$B(q) = \sum_{i=1}^n (m_{l_i} J_p^{(li)T} J_p^{(li)} + J_o^{(li)T} R_i I_i^i R_i^T J_o^{(li)}) + K_r I_m K_r$$

Where:

- m_{l_i} is the mass of the i-th link.

- $J_p^{(li)}$ is the Jacobian regarding the linear velocity of the i-th link with respect to frame 0.
- $J_o^{(li)}$ is the Jacobian regarding the angular velocity of the i-th link with respect to frame 0.
- R_i is the rotation matrix between the i-th frame and frame 0.
- $I_{l_i}^i$ is the inertia tensor of the i-th link with respect to the i-th frame.
- I_m is the inertia tensor of the i-th link motor with respect to frame 0.
- K_r is the transmission ratio matrix of the motors, it is a diagonal matrix.

These are all the parameters for the SCARA manipulator:

$$\begin{aligned}
 d_0 &= 1 \text{ m} & a_1 &= a_2 = 0.5 \text{ m} & \ell_1 &= \ell_2 = 0.25 \text{ m} \\
 m_{\ell_1} &= m_{\ell_2} = 20 \text{ kg} & m_{\ell_3} &= 10 \text{ kg} \\
 I_{\ell_1} &= I_{\ell_2} = 4 \text{ kg} \cdot \text{m}^2 & I_{\ell_4} &= 1 \text{ kg} \cdot \text{m}^2 \\
 k_{r1} &= k_{r2} = 1 & k_{r3} &= 50 \text{ rad/m} & k_{r4} &= 20 \\
 I_{m_1} &= I_{m_2} = 0.01 \text{ kg} \cdot \text{m}^2 & I_{m_3} &= 0.005 \text{ kg} \cdot \text{m}^2 & I_{m_4} &= 0.001 \text{ kg} \cdot \text{m}^2 \\
 F_{m1} &= F_{m2} = 0.00005 \text{ N} \cdot \text{m} \cdot \text{s/rad} & F_{m3} &= 0.01 \text{ N} \cdot \text{m} \cdot \text{s/rad} & F_{m4} &= 0.005 \text{ N} \cdot \text{m} \cdot \text{s/rad}
 \end{aligned}$$

Figure 6.1: SCARA manipulator datas

The non null terms of $B(q)$ are these ones:

$$b_{11} = I_{l_1} + I_{l_2} + I_{l_4} + m_{l_1} l_1^2 + m_{l_2} (a_1^2 + l_2^2 + 2a_1 l_2 \cos \vartheta_2) + m_{l_3} (a_1^2 + a_2^2 + 2a_1 a_2 \cos \vartheta_2) + k_{r1}^2 I_{m_1}$$

$$b_{22} = I_{l_2} + I_{l_4} + m_{l_2} l_2^2 + m_{l_3} a_2^2 + k_{r2}^2 I_{m_2}$$

$$b_{33} = m_{l_3} + k_{r3}^2 I_{m_3}$$

$$b_{44} = I_{l_4} + k_{r4}^2 I_{m_4}$$

$$b_{12} = b_{21} = I_{l_2} + I_{l_4} + m_{l_2} (l_2^2 + a_1 l_2 \cos \vartheta_2) + m_{l_3} (a_2^2 + a_1 a_2 \cos \vartheta_2)$$

$$b_{14} = b_{41} = b_{24} = b_{42} = -I_{l_4}$$

Then, to compute $C(q, \dot{q})$ every term of $B(q)$ is needed, so that the *Christoffel symbols* c_{ijk} can be computed to obtain c_{ij} :

$$c_{ij} = \sum_{k=1}^n c_{ijk} \dot{q}_k$$

$$c_{ijk} = \frac{1}{2} \left(\frac{\partial b_{ij}}{\partial q_k} + \frac{\partial b_{ik}}{\partial q_j} - \frac{\partial b_{jk}}{\partial q_i} \right)$$

And so, $C(q, \dot{q})$ is equal to:

$$C = \begin{bmatrix} k\dot{\vartheta}_2 & k(\dot{\vartheta}_1 + \dot{\vartheta}_2) & 0 & 0 \\ -k\dot{\vartheta}_1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Where $k = -m_{l_2} a_1 l_2 \sin(\vartheta_2) - m_{l_3} a_1 a_2 \sin(\vartheta_2)$.

F_v represents the viscous friction produced by the motors, so it can be written as:

$$F_v = \begin{bmatrix} k_{r_1}^2 F_{m_1} & 0 & 0 & 0 \\ 0 & k_{r_2}^2 F_{m_2} & 0 & 0 \\ 0 & 0 & k_{r_3}^2 F_{m_3} & 0 \\ 0 & 0 & 0 & k_{r_4}^2 F_{m_4} \end{bmatrix}$$

In the end, the gravitational terms vector $g(q)$ can be computed as:

$$g(q) = \begin{bmatrix} 0 & 0 & -g(m_{l_3} + m_p) & 0 \end{bmatrix}^T$$

Where m_p is the mass of an eventual payload.

Chapter 7

Motion Control With a Payload

Now different types of controllers will be shown: the first and the second one are in the joint space, robust and adaptive control schemes (which need a second order CLIK algorithm to be designed), while the third one is an operational space dynamic controller.

7.1 Second order CLIK algorithm with Jacobian inverse

In case a reference in terms of accelerations is needed, as well as position and velocity, it is necessary to implement a second-order CLIK algorithm with the inverse of the Jacobian. It is based on the feedback linearization technique. We compensate for nonlinear terms by a choice of $\ddot{q}$ equal to:

$$\ddot{q} = J_A^{-1}(q)(\ddot{x}_d + K_D\dot{e} + K_P e - \dot{J}_A(q, \dot{q})\dot{q})$$

Using this joint's acceleration, the following equation of the error is obtained:

$$\ddot{e} + K_D\dot{e} + K_P e = 0$$

The error system obtained is linear, time-invariant, stationary and homogeneous. Through the application of Cartesian rule, choosing K_D and K_P as positive definite matrices, the system will also be asymptotically stable. It can also be seen that it represents a PD controller with velocity and acceleration feed-forward so as to facilitate the pursuit of the trajectory. The choice of K_D and K_P matrices was as follows:

$$K_P = \begin{pmatrix} 1000 & 0 & 0 & 0 \\ 0 & 1000 & 0 & 0 \\ 0 & 0 & 1000 & 0 \\ 0 & 0 & 0 & 1000 \end{pmatrix} \quad K_D = \begin{pmatrix} 25 & 0 & 0 & 0 \\ 0 & 25 & 0 & 0 \\ 0 & 0 & 25 & 0 \\ 0 & 0 & 0 & 25 \end{pmatrix}$$

The control scheme is the following one:

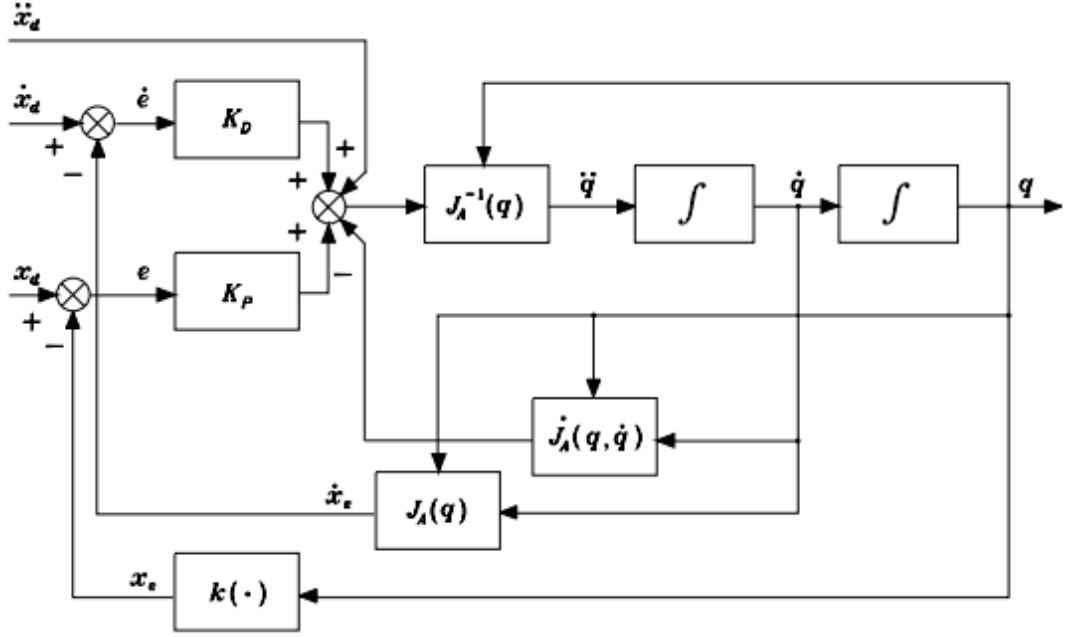


Figure 7.1: Second order CLIK algorithm

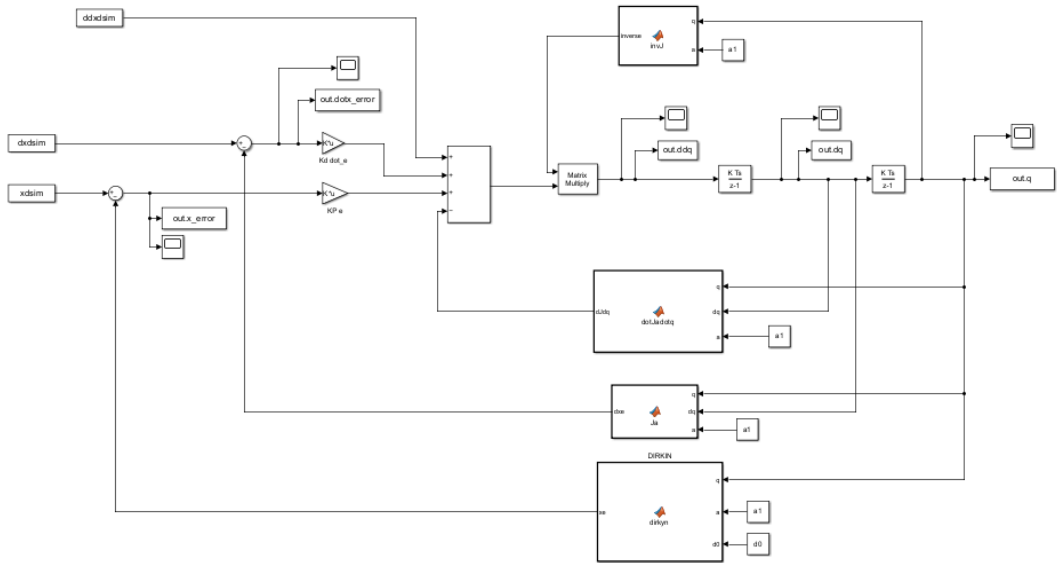


Figure 7.2: Simulink scheme of second order CLIK algorithm



Figure 7.3: Position error norm with second order CLIK algorithm

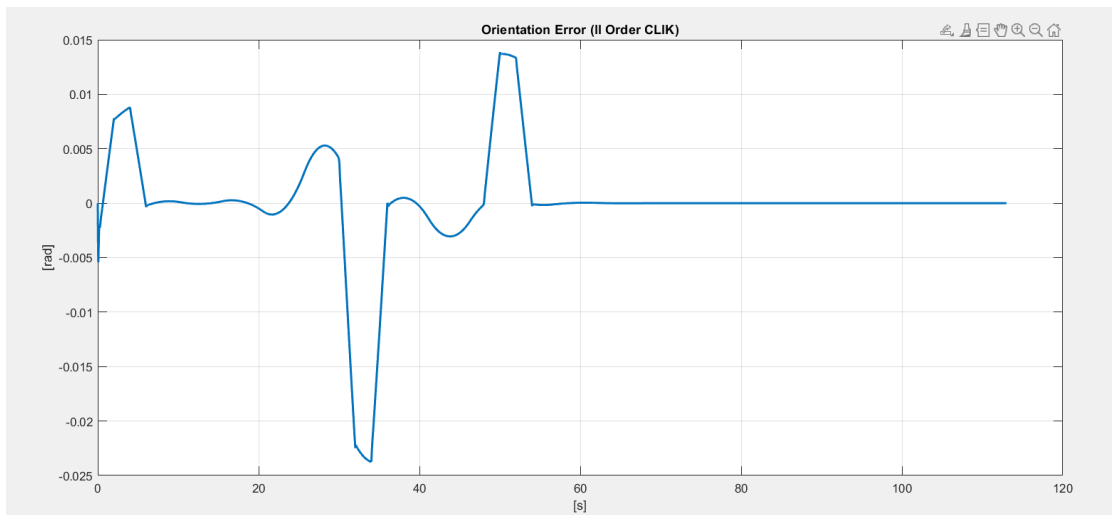


Figure 7.4: Orientation error with second order CLIK algorithm

7.2 Robust Control

Robust control is a way of compensating the error given by the approximation when computing the dynamic model. The approximation derives from the fact that the terms of the dynamic model have to be computed online and also from the fact that some term in the manipulator model may be intentionally neglected, for example the friction contribution.

In this case, the control law can be expressed as:

$$u = \hat{B}y + \hat{n}$$

Where $\hat{B}$ and $\hat{n}$ are the inaccurate expressions of the dynamic model matrices ($n = C + F_v + g$, it contains all the non linearity of the model).

It can be proven that by taking:

$$y = \ddot{q}_d + K_D \dot{\tilde{q}} + K_P \tilde{q} + w = [(I - B^{-1}\hat{B})y - B^{-1}\hat{n}] + w = \eta + w$$

$$w = \begin{cases} \frac{\rho z}{\|z\|} & \|z\| > \epsilon \\ \frac{\rho z}{\epsilon} & \|z\| \leq \epsilon \end{cases}$$

With $\rho > \|\eta\|$; this technique is called *versor control*.

The following scheme represent the robust control:

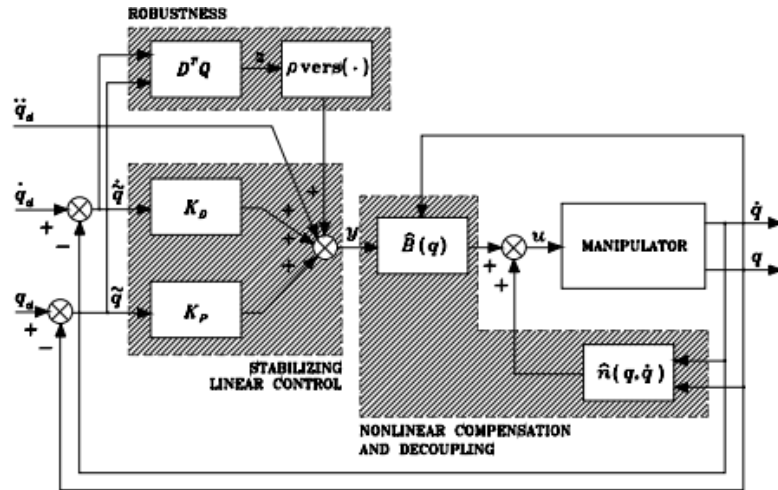


Figure 7.5: Robust control scheme

ρ is the gain constant, while z is defined as:

$$z = D^T Q \xi$$

$$D = \begin{bmatrix} O \\ I \end{bmatrix} \quad \xi = \begin{bmatrix} \tilde{q} \\ \dot{\tilde{q}} \end{bmatrix}$$

And Q is a square positive definite matrix, with dimensions $2n \times 2n$, with n number of joints, therefore ξ is a $2n \times 1$ vector, and D is a $2n \times n$ matrix.

This versor control law brings z to zero, which, being bound to ξ , means bringing position and velocity errors to zero. With a diagram in the state space we have:

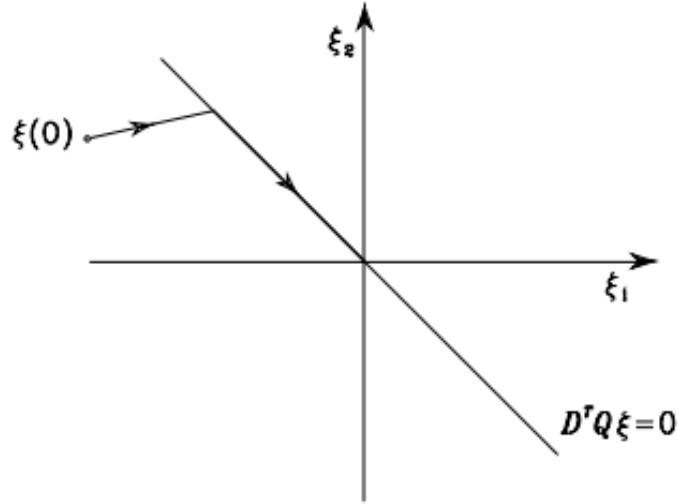


Figure 7.6: Space state diagram

The control works in such a way as to attract trajectories towards $z = 0$; this means that the control oscillates at high frequency.

In this scheme there is a problem; if we start from 0 error, the system abhors, w gets close to infinity, and so a threshold is needed, to saturate the controller; this threshold is ϵ :

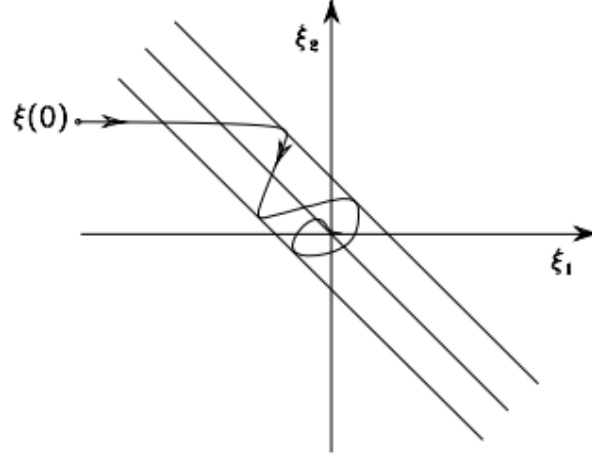


Figure 7.7: Space state diagram with boundary layer

With these scheme, the trajectories will evolve into the boundary layer until they converge into the origin of the space state.

In the simulation these parameters were used:

$$\rho = 70$$

$$K_P = \begin{pmatrix} 400 & 0 & 0 & 0 \\ 0 & 400 & 0 & 0 \\ 0 & 0 & 400 & 0 \\ 0 & 0 & 0 & 400 \end{pmatrix} \quad K_D = \begin{pmatrix} 25 & 0 & 0 & 0 \\ 0 & 25 & 0 & 0 \\ 0 & 0 & 25 & 0 \\ 0 & 0 & 0 & 25 \end{pmatrix}$$

$$Q = \begin{pmatrix} 200 & 0 & 0 & 0 & 200 & 0 & 0 & 0 \\ 0 & 200 & 0 & 0 & 0 & 200 & 0 & 0 \\ 0 & 0 & 200 & 0 & 0 & 0 & 200 & 0 \\ 0 & 0 & 0 & 200 & 0 & 0 & 0 & 200 \\ 200 & 0 & 0 & 0 & 0.1 & 0 & 0 & 0 \\ 0 & 200 & 0 & 0 & 0 & 0.1 & 0 & 0 \\ 0 & 0 & 200 & 0 & 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 200 & 0 & 0 & 0 & 0.1 \end{pmatrix}$$

This choice allows to have a greater weight on the position error than on the velocity one, as it's better to have a velocity error that results in a delay compared to having an error in position because in this way there is the security of following the right trajectory.

The implemented control scheme is as follows:

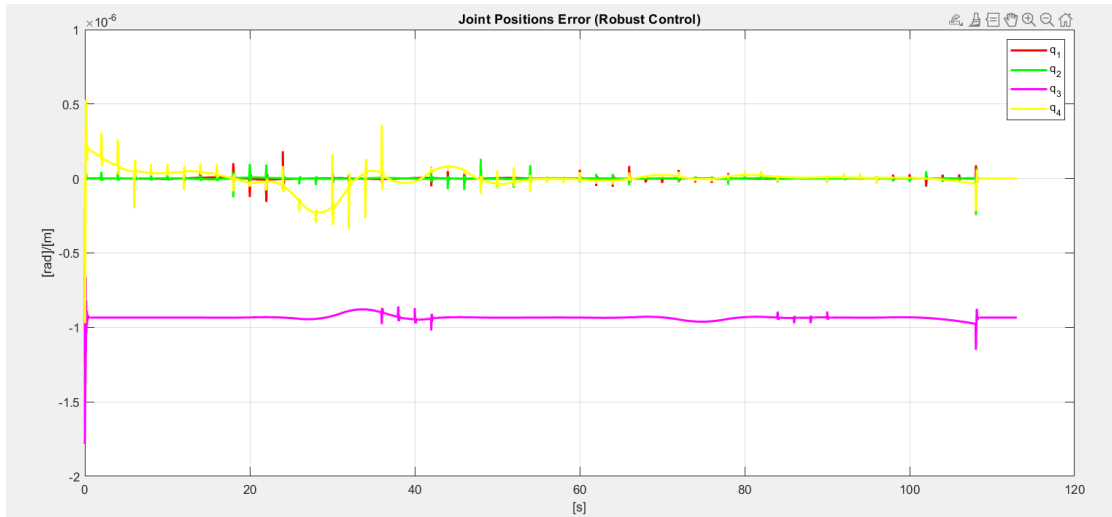


Figure 7.10: Joint position error for robust control

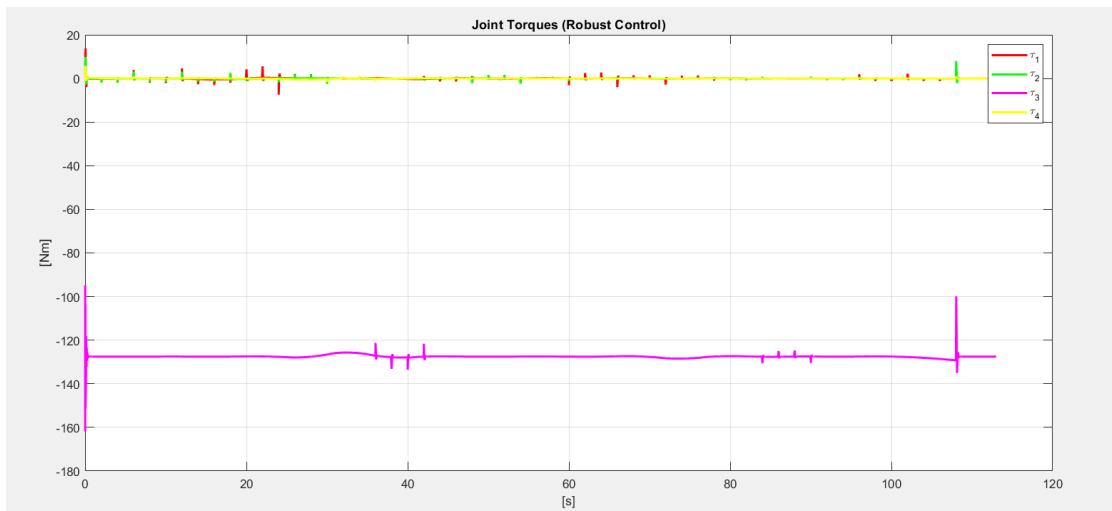


Figure 7.11: Joint torques with robust control ($\epsilon = 0.01$)

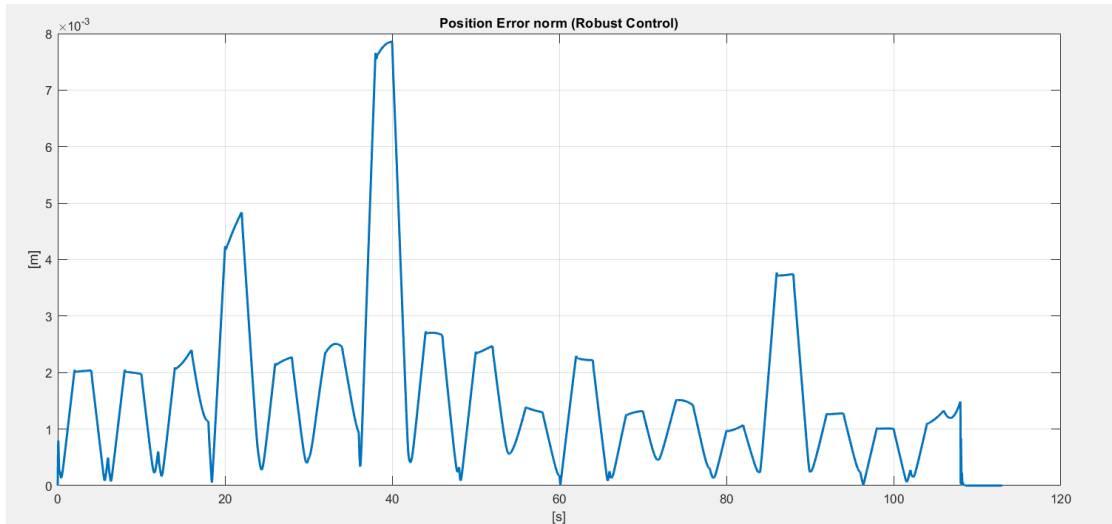


Figure 7.12: Position error norm with robust control

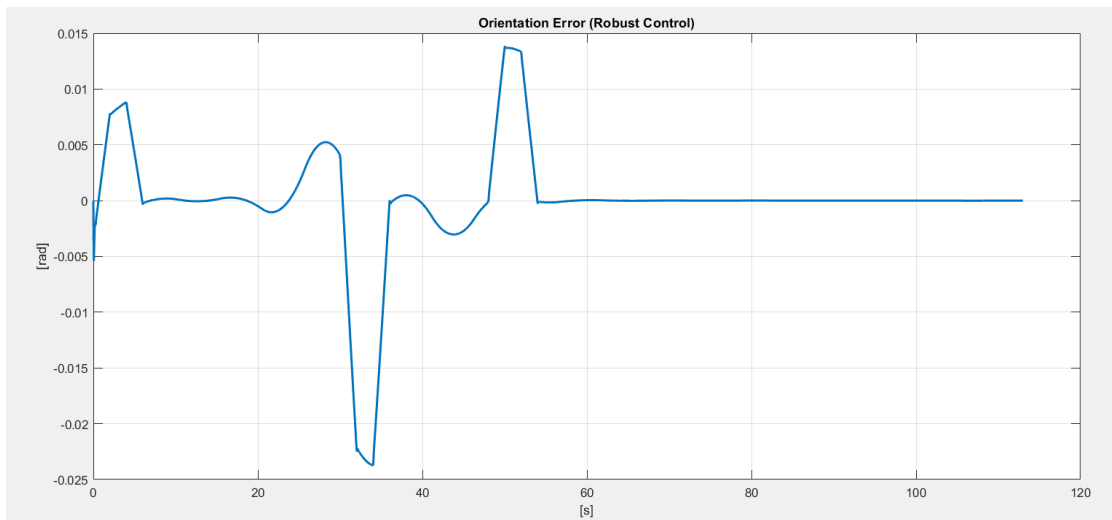
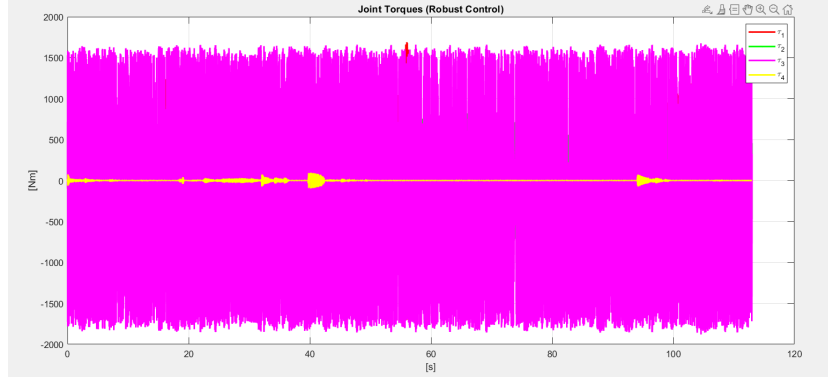


Figure 7.13: Orientation error with robust control

If $\epsilon = 0.001$ a phenomenon called *chattering* appears, due to fluctuations at high frequency of the control.


 Figure 7.14: Joint torques with robust control ($\epsilon = 0.001$)

7.3 Adaptive Control

While robust control acts on the structure of the controller, adaptive control will act on the parameters of the model, making up for the possibility that dynamic parameters might be known with some uncertainty, like when we have a payload. It exploits the property that the dynamic model can be written as a system of linear equations respect to the parameters:

$$B(q)\ddot{q} + C(q, \dot{q})\dot{q} + F_v\dot{q} + g(q) = Y(q, \dot{q}, \ddot{q})\pi = u$$

Where Y is an $n \times p$ matrix (n number of joints and p number of dynamic parameters) called *regressor*.

π is the dynamic parameters vector, $p \times 1$:

$$\pi = [m_{l_1} \ I_{l_1} \ I_{m_1} \ F_{m_1} \ m_{l_2} \ I_{l_2} \ I_{m_2} \ F_{m_2} \ m_{l_3} \ I_{m_3} \ F_{m_3} \ I_{l_4} \ I_{m_4} \ F_{m_4}]$$

The control input will be:

$$u = B(q)\ddot{q}_r + C(q, \dot{q})\dot{q}_r + F_v\dot{q}_r + g(q) + K_D\sigma$$

Where:

- $\ddot{q}_r = \ddot{q}_d + \Lambda\dot{\tilde{q}}$.
- $\dot{q}_r = \dot{q}_d + \Lambda\tilde{q}$.
- Λ is a positive definite matrix.
- $\sigma = \dot{\tilde{q}} + \Lambda\tilde{q}$

In the case of control based on parameter estimation, the input becomes:

$$u = Y(q, \dot{q}, \dot{q}_r, \ddot{q}_r) \hat{\pi} + K_D \sigma$$

Where $\hat{\pi}$ is a vector that contains an estimation of the parameters.

In this case, the uncertainty does not relate to the analytical structure of the terms of the model (the matrix Y remains the same), but only affects the parameters. By integrating the law of adaptation on the parameters we obtain $\hat{\pi}$:

$$\dot{\hat{\pi}} = K_{\pi}^{-1} Y^T \sigma$$

Where K_{π} is a diagonal matrix of gains. The regressor matrix is calculated as $Y = \frac{\partial \tau}{\partial \pi}$. For the simulation, this choices were made:

$$K_D = \begin{pmatrix} 950 & 0 & 0 & 0 \\ 0 & 950 & 0 & 0 \\ 0 & 0 & 950 & 0 \\ 0 & 0 & 0 & 950 \end{pmatrix} \quad \Lambda = \begin{pmatrix} 50 & 0 & 0 & 0 \\ 0 & 50 & 0 & 0 \\ 0 & 0 & 50 & 0 \\ 0 & 0 & 0 & 50 \end{pmatrix}$$

$$K_{\pi} = \text{diag}\{100, 100, 100, 100, 100, 100, 100, 100, 0.01, 100, 100, 100, 100, 100\}$$

Since the mass m_{l_3} is the parameter affected by uncertainty, the gain of the action will have a greater weight at that parameter. The scheme implemented for this control is:

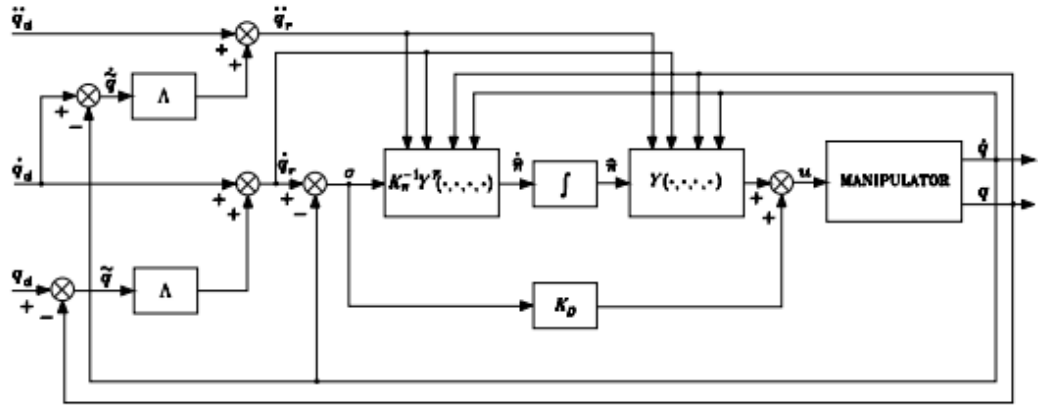


Figure 7.15: Adaptive control scheme

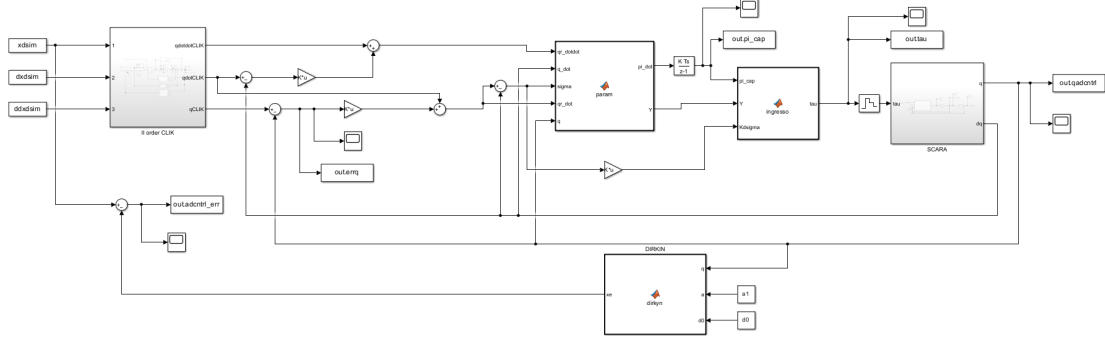


Figure 7.16: Simulink scheme for adaptive control

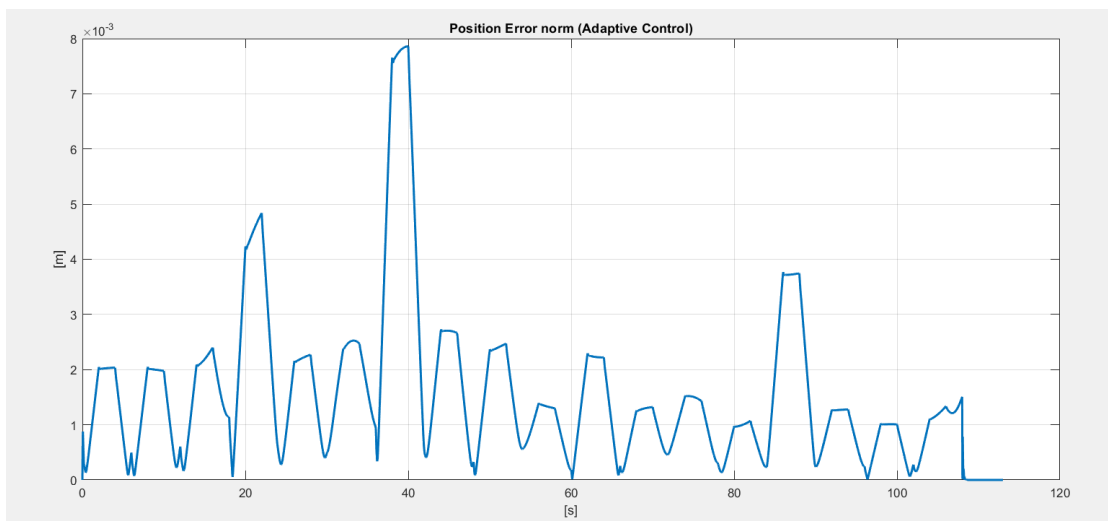


Figure 7.17: Position error norm with adaptive control

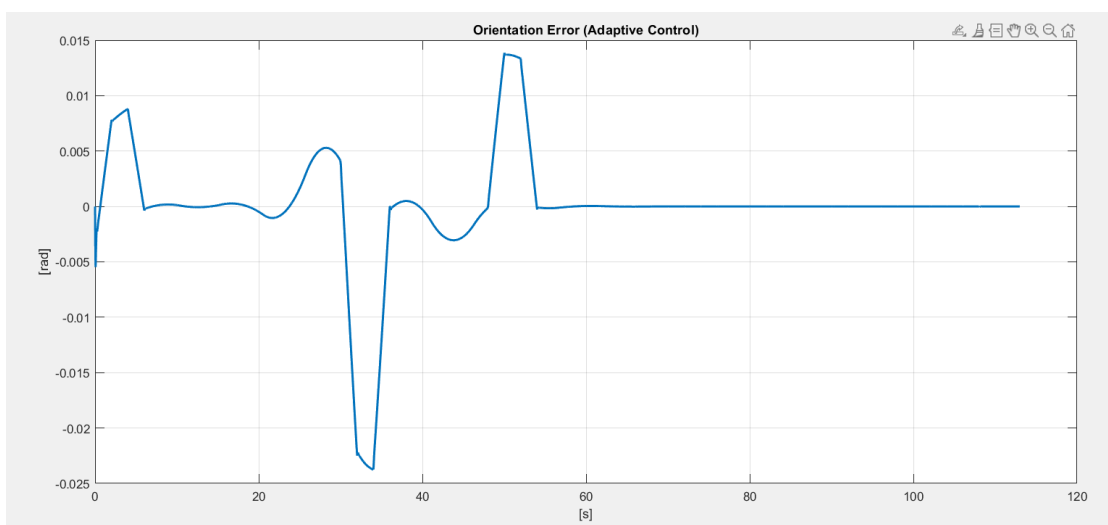


Figure 7.18: Orientation error with adaptive control

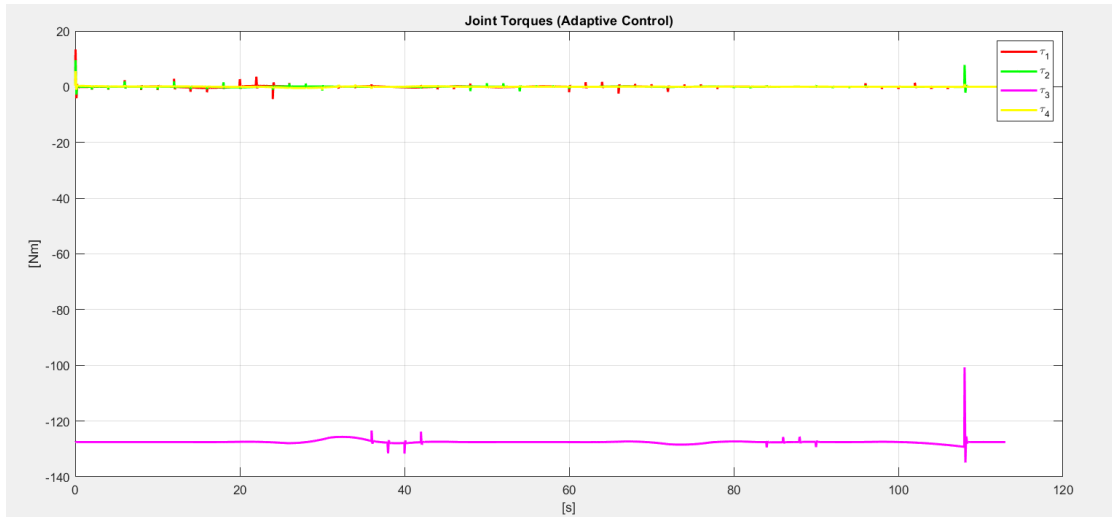


Figure 7.19: Joint torques with adaptive control

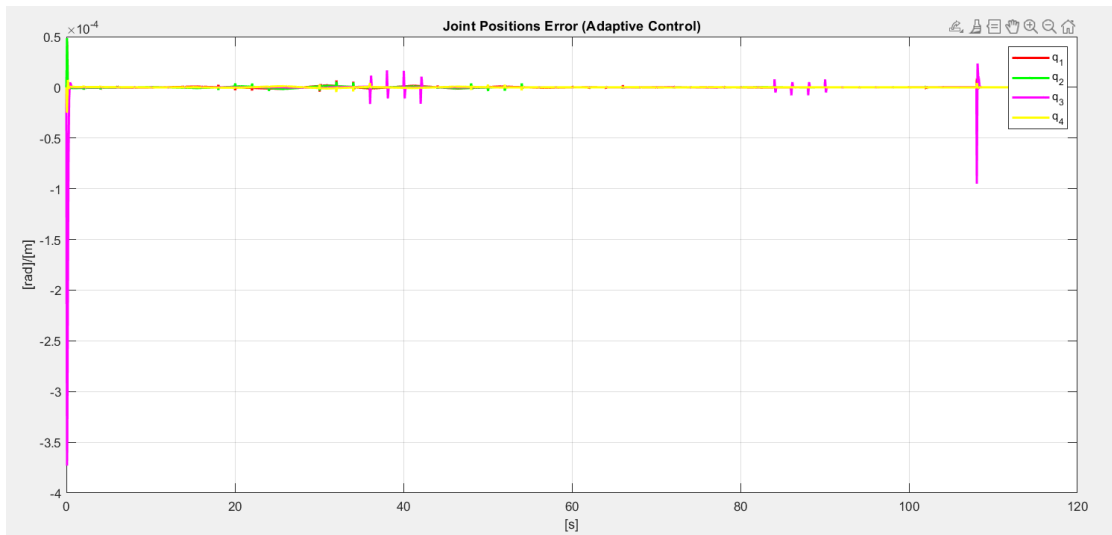


Figure 7.20: Joint position error with adaptive control

7.4 Inverse Dynamics Control in Operational Space

Up to this point, the control schemes have been realized into joint's space, using algorithms for kinematic inversion. Now an inverse dynamic control in the operational will be designed.

Now the input of the controller will be a desired trajectory of the end effector into the operational space, x_d , with its velocity and acceleration.

The input is chosen as:

$$u = By + n$$

$$y = J_A^{-1}(\ddot{x}_d + K_D\dot{\tilde{x}} + K_P\tilde{x} + K_I \int \tilde{x}dt - \dot{J}_A\dot{q})$$

An integral action is added to recover the steady state error caused by the payload.

The matrices K_D, K_P, K_I were chosen like this:

$$K_P = \begin{pmatrix} 2500 & 0 & 0 & 0 \\ 0 & 2500 & 0 & 0 \\ 0 & 0 & 2500 & 0 \\ 0 & 0 & 0 & 2500 \end{pmatrix} \quad K_D = \begin{pmatrix} 80 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 \\ 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 80 \end{pmatrix}$$

$$K_I = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Here is reported the scheme of the inverse dynamics controller in the operational space:

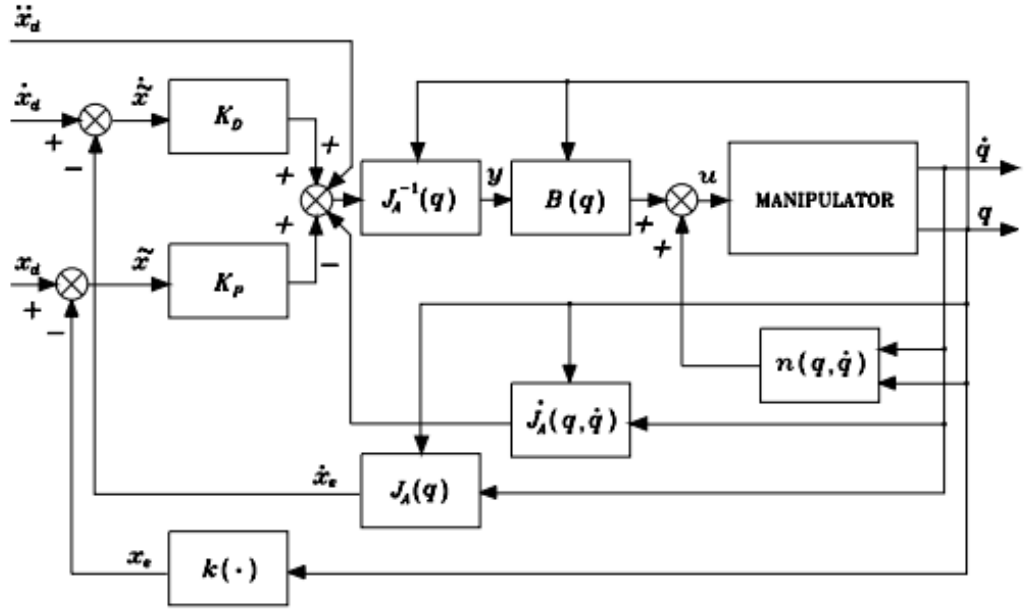


Figure 7.21: Inverse dynamics block scheme

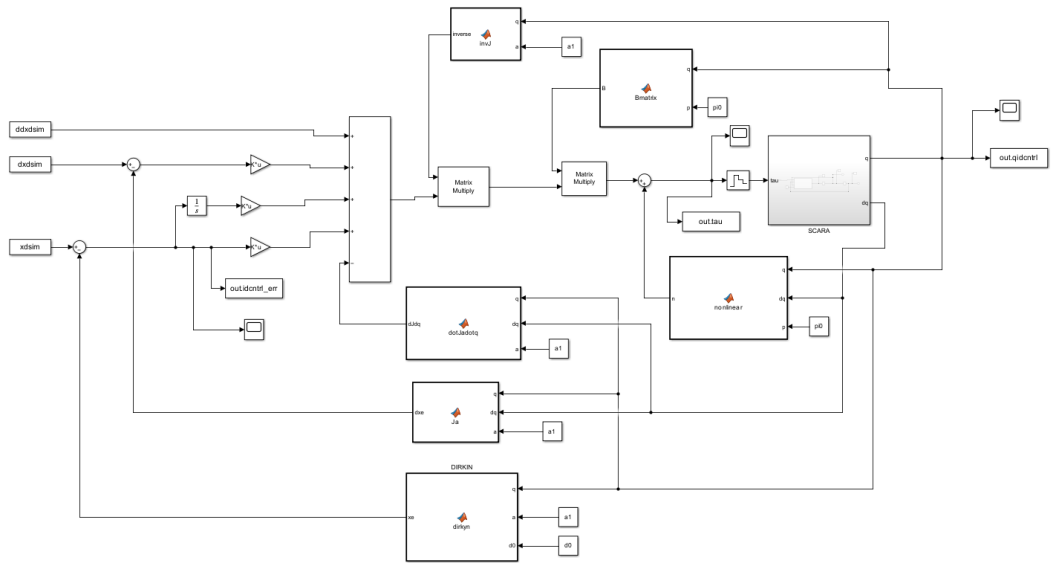


Figure 7.22: Simulink scheme for inverse dynamics control

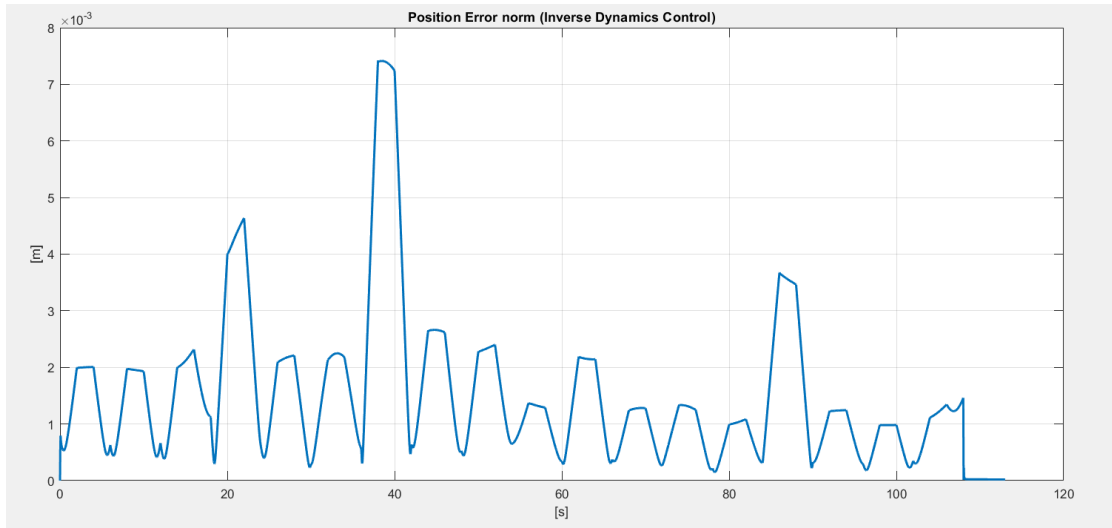


Figure 7.23: Position error norm with inverse dynamics control

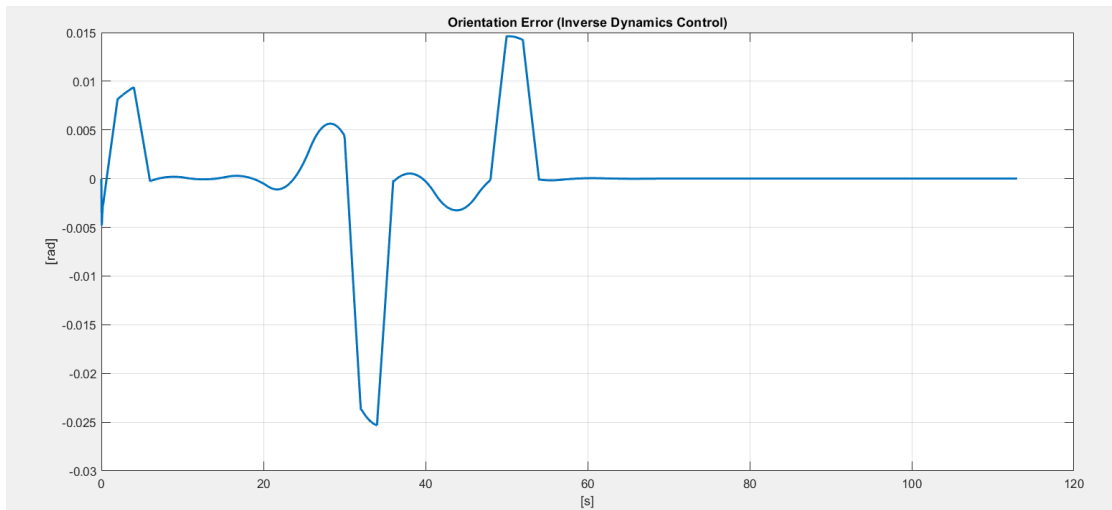


Figure 7.24: Orientation error with inverse dynamics control

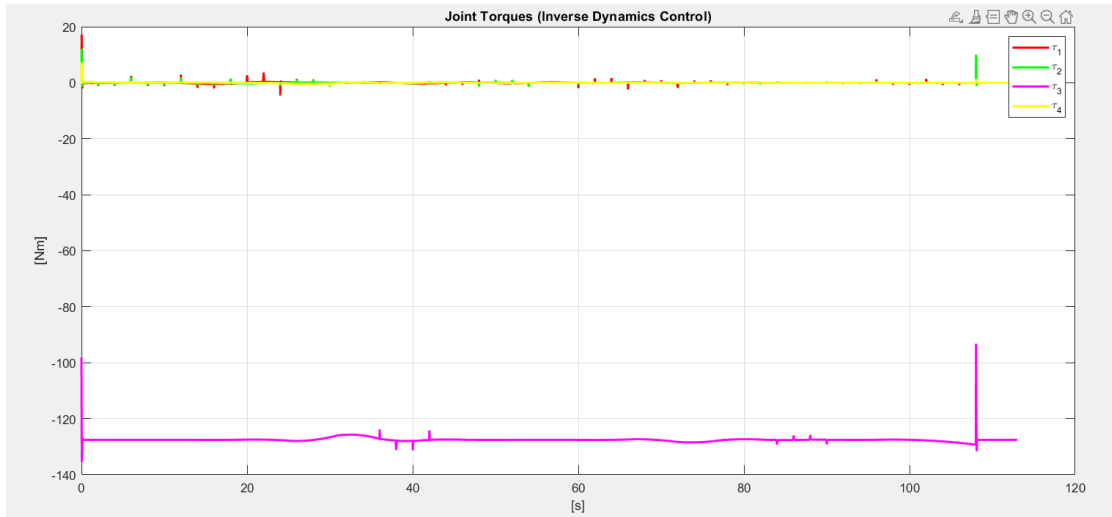


Figure 7.25: Joint torques with inverse dynamics control